Program 3.13 List allocation for the Josephus problem

This program for the Josephus problem is an example of a client program utilizing the list-processing primitives declared in Program 3.12 and implemented in Program 3.14.

```
#include "list.h"
main(int argc, char *argv[])
  { int i, N = atoi(argv[1]), M = atoi(argv[2]);
    Node t, x;
    initNodes(N);
    for (i = 2, x = newNode(1); i <= N; i++)
      { t = newNode(i); insertNext(x, t); x = t; }
    while (x != Next(x))
      {
        for (i = 1; i < M; i++) x = Next(x);
        freeNode(deleteNext(x));
      }
    printf("%d\n", Item(x));
  }
```

it leaves an empty list. One such mechanism—the one used for the function in Program 3.10—is to have list-processing functions take pointers to input lists as arguments and return pointers to output lists. With this convention, we do not need to use head nodes. Furthermore, it is well suited to recursive list processing, which we use extensively throughout the book (see Section 5.1).

Program 3.12 illustrates declarations for a set of black-box functions that implement the basic list operations, in case we choose not to repeat the code inline. Program 3.13 is our Josephus-election program (Program 3.9) recast as a client program that uses this interface. Identifying the important operations that we use in a computation and defining them in an interface gives us the flexibility to consider different concrete implementations of critical operations and to test their effectiveness. We consider one implementation for the operations defined in Program 3.12 in Section 3.5 (see Program 3.14), but we could also try other alternatives without changing Program 3.13 at all (see Exercise 3.52). This theme will recur throughout the book,

and we will consider mechanisms to make it easier to develop such implementations in Chapter 4.

Some programmers prefer to encapsulate all operations on low-level data structures such as linked lists by defining functions for every low-level operation in interfaces like Program 3.12. Indeed, as we shall see in Chapter 4, the C class mechanism makes it easy to do so. However, that extra layer of abstraction sometimes masks the fact that just a few low-level operations are involved. In this book, when we are implementing higher-level interfaces, we usually write low-level operations on linked structures directly, to clearly expose the essential details of our algorithms and data structures. We shall see many examples in Chapter 4.

By adding more links, we can add the capability to move backward through a linked list. For example, we can support the operation "find the item *before* a given item" by using a *doubly linked list* in which we maintain two links for each node: one (`prev`) to the item before, and another (`next`) to the item after. With dummy nodes or a circular list, we can ensure that `x`, `x->next->prev`, and `x->prev->next` are the same for every node in a doubly linked list. Figures 3.9 and 3.10 show the basic link manipulations required to implement *delete*, *insert after*, and *insert before*, in a doubly linked list. Note that, for *delete*, we do not need extra information about the node before it (or the node after it) in the list, as we did for singly linked lists—that information is contained in the node itself.

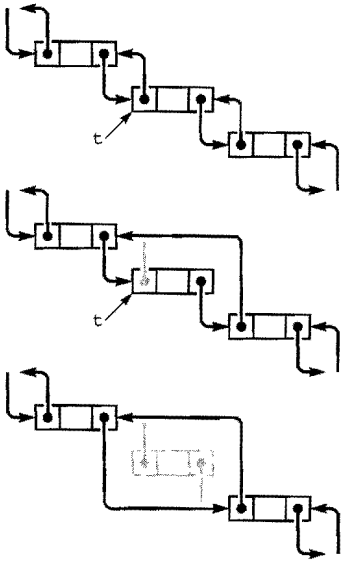


Figure 3.9
Deletion in a doubly-linked list

In a doubly-linked list, a pointer to a node is sufficient information for us to be able to remove it, as diagrammed here. Given `t`, *we set* `t->next->prev` *to* `t->prev` (center) *and* `t->prev->next` *to* `t->next` (bottom).

Indeed, the primary significance of doubly linked lists is that they allow us to delete a node when the *only* information that we have about that node is a link to it. Typical situations are when the link is passed as an argument in a function call, and when the node has other links and is also part of some other data structure. Providing this extra capability doubles the space needed for links in each node and doubles the number of link manipulations per basic operation, so doubly linked lists are not normally used unless specifically called for. We defer considering detailed implementations to a few specific situations where we have such a need—for example in Section 9.5.

We use linked lists throughout this book, first for basic ADT implementations (see Chapter 4), then as components in more complex data structures. Linked lists are many programmers' first exposure to an abstract data structure that is under the programmers' direct

control. They represent an essential tool for our use in developing the high-level abstract data structures that we need for a host of important problems, as we shall see.

Exercises

▷ 3.34 Write a function that moves the largest item on a given list to be the final node on the list.

3.35 Write a function that moves the smallest item on a given list to be the first node on the list.

3.36 Write a function that rearranges a linked list to put the nodes in even positions after the nodes in odd positions in the list, preserving the relative order of both the evens and the odds.

3.37 Implement a code fragment for a linked list that exchanges the positions of the nodes after the nodes referenced by two given links `t` and `u`.

○ 3.38 Write a function that takes a link to a list as argument and returns a link to a copy of the list (a new list that contains the same items, in the same order).

3.39 Write a function that takes two arguments—a link to a list and a function that takes a link as argument—and removes all items on the given list for which the function returns a nonzero value.

3.40 Solve Exercise 3.39, but make copies of the nodes that pass the test and return a link to a list containing those nodes, in the order that they appear in the original list.

3.41 Implement a version of Program 3.10 that uses a head node.

3.42 Implement a version of Program 3.11 that does not use head nodes.

3.43 Implement a version of Program 3.9 that uses a head node.

3.44 Implement a function that exchanges two given nodes on a doubly linked list.

○ 3.45 Give an entry for Table 3.1 for a list that is never empty, is referred to with a pointer to the first node, and for which the final node has a pointer to itself.

3.46 Give an entry for Table 3.1 for a circular list that has a dummy node, which serves as both head and tail.

3.5 Memory Allocation for Lists

An advantage of linked lists over arrays is that linked lists gracefully grow and shrink during their lifetime. In particular, their maximum

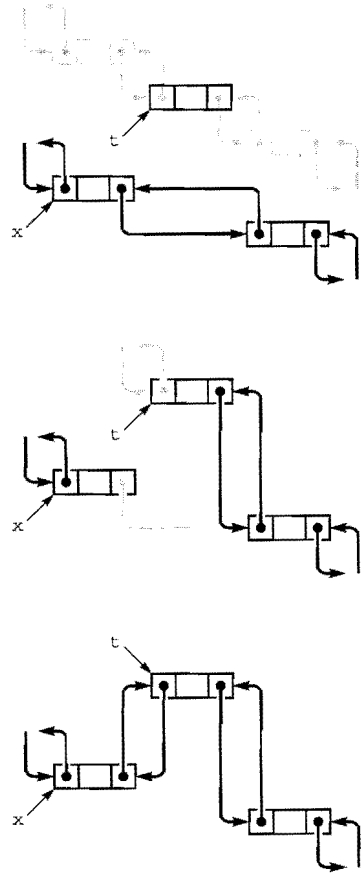


Figure 3.10
Insertion in a doubly-linked list

To insert a node into a doubly-linked list, we need to set four pointers. We can insert a new node after a given node (diagrammed here) or before a given node. We insert a given node `t` *after another given node* `x` *by setting* `t->next` *to* `x->next` *and* `x->next->prev` *to* `t` (center), *and then setting* `x->next` *to* `t` *and* `t->prev` *to* `x` (bottom).

size does not need to be known in advance. One important practical ramification of this observation is that we can have several data structures share the same space, without paying particular attention to their relative size at any time.

The crux of the matter is to consider how the system function `malloc` might be implemented. For example, when we delete a node from a list, it is one thing for us to rearrange the links so that the node is no longer hooked into the list, but what does the system do with the space that the node occupied? And how does the system recycle space such that it can always find space for a node when `malloc` is called and more space is needed? The mechanisms behind these questions provide another example of the utility of elementary list processing.

The system function `free` is the counterpart to `malloc`. When we are done using a chunk of allocated memory, we call `free` to inform the system that the chunk is available for later use. *Dynamic memory allocation* is the process of managing memory and responding to calls on `malloc` and `free` from client programs.

When we are calling `malloc` directly in applications such as Program 3.9 or Program 3.11, all the calls request memory blocks of the same size. This case is typical, and an alternate method of keeping track of memory available for allocation immediately suggests itself: Simply use a linked list! All nodes that are not on any list that is in use can be kept together on a single linked list. We refer to this list as the *free list*. When we need to allocate space for a node, we get it by *deleting* it from the free list; when we remove a node from any of our lists, we dispose of it by *inserting* it onto the free list.

Program 3.14 is an implementation of the interface defined in Program 3.12, including the memory-allocation functions. When compiled with Program 3.13, it produces the same result as the direct implementation with which we began in Program 3.9. Maintaining the free list for fixed-size nodes is a trivial task, given the basic operations for inserting nodes onto and deleting nodes from a list.

Figure 3.11 illustrates how the free list grows as nodes are freed, for Program 3.13. For simplicity, the figure assumes a linked-list implementation (no head node) based on array indices.

Implementing a general-purpose memory allocator in a C environment is much more complex than is suggested by our simple examples, and the implementation of `malloc` in the standard library

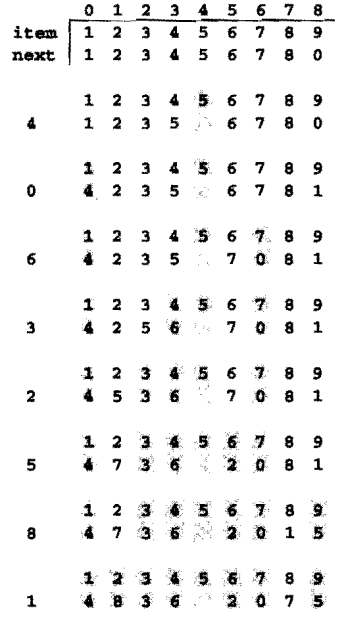


Figure 3.11
Array representation of a linked list, with free list

This version of Figure 3.6 shows the result of maintaining a free list with the nodes deleted from the circular list, with the index of first node on the free list given at the left. At the end of the process, the free list is a linked list containing all the items that were deleted. Following the links, starting at 1, *we see the items in the order* 2 9 6 3 4 7 1 5, *which is the reverse of the order in which they were deleted.*

Program 3.14 Implementation of list-processing interface

This program gives implementations of the functions declared in Program 3.12, and illustrates a standard approach to allocating memory for fixed-size nodes. We build a free list that is initialized to the maximum number of nodes that our program will use, all linked together. Then, when a client program allocates a node, we remove that node from the free list; when a client program frees a node, we link that node in to the free list.

By convention, client programs do not refer to list nodes except through function calls, and nodes returned to client programs have self-links. These conventions provide some measure of protection against referencing undefined pointers.

```
#include <stdlib.h>
#include "list.h"
link freelist;
void initNodes(int N)
  { int i;
    freelist = malloc((N+1)*(sizeof *freelist));
    for (i = 0; i < N+1; i++)
      freelist[i].next = &freelist[i+1];
    freelist[N].next = NULL;
  }
link newNode(int i)
  { link x = deleteNext(freelist);
    x->item = i; x->next = x;
    return x;
  }
void freeNode(link x)
  { insertNext(freelist, x); }
void insertNext(link x, link t)
  { t->next = x->next; x->next = t; }
link deleteNext(link x)
  { link t = x->next; x->next = t->next; return t; }
link Next(link x)
  { return x->next; }
int Item(link x)
  { return x->item; }
```

is certainly not as simple as is indicated by Program 3.14. One primary difference between the two is that `malloc` has to handle storage-allocation requests for nodes of varying sizes, ranging from tiny to huge. Several clever algorithms have been developed for this purpose. Another approach that is used by some modern systems is to relieve the user of the need to `free` nodes explicitly by using *garbage-collection* algorithms to remove automatically any nodes not referenced by any link. Several clever storage management algorithms have also been developed along these lines. We will not consider them in further detail because their performance characteristics are dependent on properties of specific systems and machines.

Programs that can take advantage of specialized knowledge about an application often are more efficient than general-purpose programs for the same task. Memory allocation is no exception to this maxim. An algorithm that has to handle storage requests of varying sizes cannot know that we are always going to be making requests for blocks of one fixed size, and therefore cannot take advantage of that fact. Paradoxically, another reason to avoid general-purpose library functions is that doing so makes programs more portable—we can protect ourselves against unexpected performance changes when the library changes or when we move to a different system. Many programmers have found that using a simple memory allocator like the one illustrated in Program 3.14 is an effective way to develop efficient and portable programs that use linked lists. This approach applies to a number of the algorithms that we will consider throughout this book, which make similar kinds of demands on the memory-management system.

Exercises

○ 3.47 Write a program that frees (calls `free` with a pointer to) all the nodes on a given linked list.

3.48 Write a program that frees the nodes in positions that are divisible by 5 in a linked list (the fifth, tenth, fifteenth, and so forth).

○ 3.49 Write a program that frees the nodes in even positions in a linked list (the second, fourth, sixth, and so forth).

3.50 Implement the interface in Program 3.12 using `malloc` and `free` directly in `allocNode` and `freeNode`, respectively.

3.51 Run empirical studies comparing the running times of the memory-allocation functions in Program 3.14 with `malloc` and `free` (see Exer-

cise 3.50) for Program 3.13 with $M = 2$ and $N = 10^3$, 10^4, 10^5, and 10^6.

3.52 Implement the interface in Program 3.12 using array indices (and no head node) rather than pointers, in such a way that Figure 3.11 is a trace of the operation of your program.

○ 3.53 Suppose that you have a set of nodes with no null pointers (each node points to itself or to some other node in the set). Prove that you ultimately get into a cycle if you start at any given node and follow links.

• 3.54 Under the conditions of Exercise 3.53, write a code fragment that, given a pointer to a node, finds the number of different nodes that it ultimately reaches by following links from that node, *without* modifying any nodes. Do not use more than a constant amount of extra memory space.

•• 3.55 Under the conditions of Exercise 3.54, write a function that determines whether or not two given links, if followed, eventually end up on the same cycle.

3.6 Strings

We use the term *string* to refer to a variable-length array of characters, defined by a starting point and by a string-termination character marking the end. Strings are valuable as low-level data structures, for two basic reasons. First, many computing applications involve processing textual data, which can be represented directly with strings. Second, many computer systems provide direct and efficient access to *bytes* of memory, which correspond directly to characters in strings. That is, in a great many situations, the string abstraction matches needs of the application to the capabilities of the machine.

The abstract notion of a sequence of characters ending with a string-termination character could be implemented in many ways. For example, we could use a linked list, although that choice would exact a cost of one pointer per character. The concrete array-based implementation that we consider in this section is the one that is built into C. We shall also examine other implementations in Chapter 4.

The difference between a string and an array of characters revolves around *length*. Both represent contiguous areas of memory, but the length of an array is set at the time that the array is created, whereas the length of a string may change during the execution of a program. This difference has interesting implications, which we shall explore shortly.

We need to reserve memory for a string, either at compile time, by declaring a fixed-length array of characters, or at execution time, by calling `malloc`. Once the array is allocated, we can fill it with characters, starting at the beginning, and ending with the string-termination character. Without a string-termination character, a string is no more or no less than an array of characters; with the string-termination character, we can work at a higher level of abstraction, and consider only the portion of the array from the beginning to the string-termination character to contain meaningful information. In C, the termination character is the one with value 0, also known as `'\0'`.

For example, to find the length of a string, we count the number of characters between the beginning and the string-termination character. Table 3.2 gives simple operations that we commonly perform on strings. They all involve processing the strings by scanning through them from beginning to end. Many of these functions are available as library functions declared in `<string.h>`, although many programmers use slightly modified versions in inline code for simple applications. Robust functions implementing the same operations would have extra code to check for error conditions. We include the code here not just to highlight its simplicity, but also to expose its performance characteristics plainly.

One of the most important operations that we perform on strings is the *compare* operation, which tells us which of two strings would appear first in the dictionary. For purposes of discussion, we assume an idealized dictionary (since the actual rules for strings that contain punctuation, uppercase and lowercase letters, numbers, and so forth are rather complex), and compare strings character-by-character, from beginning to end. This ordering is called *lexicographic order*. We also use the compare function to tell whether strings are equal—by convention, the compare function returns a negative number if the first argument string appears before the second in the dictionary, returns 0 if they are equal, and returns 1 if the first appears after the second in lexicographic order. It is critical to take note that doing equality testing is *not* the same as determining whether two string *pointers* are equal—if two string pointers are equal, then so are the referenced strings (they are the *same* string), but we also could have different string pointers that point to equal strings (identical sequences of characters). Numerous applications involve storing information as strings, then processing

Table 3.2 Elementary string-processing operations

This table gives implementations of basic string-processing operations, using two different C language primitives. The pointer approach leads to more compact code, but the indexed-array approach is a more natural way to express the algorithms and leads to code that is easier to understand. The pointer version of the concatenate operation is the same as the indexed array version, and the pointer version of prefixed compare is obtained from the normal compare in the same way as for the indexed array version and is omitted. The implementations all take time proportional to string lengths.

Indexed array versions

Compute string length (`strlen(a)`)

```
for (i = 0; a[i] != 0; i++) ; return i;
```

Copy (`strcpy(a, b)`)

```
for (i = 0; (a[i] = b[i]) != 0; i++) ;
```

Compare (`strcmp(a, b)`)

```
for (i = 0; a[i] == b[i]; i++)
    if (a[i] == 0) return 0;
return a[i] - b[i];
```

Compare (prefix) (`strncmp(a, b, strlen(a))`)

```
for (i = 0; a[i] == b[i]; i++)
    if (a[i] == 0) return 0;
if (a[i] == 0) return 0;
return a[i] - b[i];
```

Append (`strcat(a, b)`)

```
strcpy(a+strlen(a), b)
```

Equivalent pointer versions

Compute string length (`strlen(a)`)

```
b = a; while (*b++) ; return b-a-1;
```

Copy (`strcpy(a, b)`)

```
while (*a++ = *b++) ;
```

Compare (`strcmp(a, b)`)

```
while (*a++ == *b++)
    if (*(a-1) == 0) return 0;
return *(a-1) - *(b-1);
```

Program 3.15 String search

This program discovers all occurrences of a word from the command line in a (presumably much larger) text string. We declare the text string as a fixed-size character array (we could also use `malloc`, as in Program 3.6) and read it from standard input, using `getchar()`. Memory for the word from the command line-argument is allocated by the system before this program is invoked, and we find the string pointer in `argv[1]`. For each starting position `i` in `a`, we try matching the substring starting at that position with `p`, testing for equality character by character. Whenever we reach the end of `p` successfully, we print out the starting position `i` of the occurrence of the word in the text.

```
#include <stdio.h>
#define N 10000
main(int argc, char *argv[])
  { int i, j, t;
    char a[N], *p = argv[1];
    for (i = 0; i < N-1; a[i] = t, i++)
      if ((t = getchar()) == EOF) break;
    a[i] = 0;
    for (i = 0; a[i] != 0; i++)
      {
        for (j = 0; p[j] != 0; j++)
          if (a[i+j] != p[j]) break;
        if (p[j] == 0) printf("%d ", i);
      }
    printf("\n");
  }
```

or accessing that information by comparing the strings, so the compare operation is a particularly critical one. We shall see a specific example in Section 3.7 and in numerous other places throughout the book.

Program 3.15 is an implementation of a simple string-processing task, which prints out the places where a short pattern string appears within a long text string. Several sophisticated algorithms have been developed for this task, but this simple one illustrates several of the conventions that we use when processing strings in C.

String processing provides a convincing example of the need to be knowledgeable about the performance of library functions. The

problem is that a library function might take more time than we expect, intuitively. For example, *determining the length of a string takes time proportional to the length of the string*. Ignoring this fact can lead to severe performance problems. For example, after a quick look at the library, we might implement the pattern match in Program 3.15 as follows:

```
for (i = 0; i < strlen(a); i++)
  if (strncmp(&a[i], p, strlen(p)) == 0)
    printf("%d ", i);
```

Unfortunately, this code fragment takes time proportional to at least the *square* of the length of a, no matter what code is in the body of the loop, because it goes all the way through a to determine its length each time through the loop. This cost is considerable, even prohibitive: Running this program to check whether this book (which has more than 1 million characters) contains a certain word would require trillions of instructions. Problems such as this one are difficult to detect because the program might work fine when we are debugging it for small strings, but then slow down or even never finish when it goes into production. Moreover, we can avoid such problems only if we know about them!

This kind of error is called a *performance bug*, because the code can be verified to be correct, but it does not perform as efficiently as we (implicitly) expect. Before we can even begin the study of efficient algorithms, we must be certain to have eliminated performance bugs of this type. Although standard libraries have many virtues, we must be wary of the dangers of using them for simple functions of this kind.

One of the essential concepts that we return to time and again in this book is that different implementations of the same abstract notion can lead to widely different performance characteristics. For example, if we keep track of the length of the string, we can support a function that can return the length of a string in constant time, but for which other operations run more slowly. One implementation might be appropriate for one application; another implementation might be appropriate for another application.

Library functions, all too often, cannot guarantee to provide the best performance for all applications. Even if (as in the case of `strlen`) the performance of a library function is well documented, we have no assurance that some future implementation might not involve

performance changes that will have adverse effects on our programs. This issue is critical in the design of algorithms and data structures, and thus is one that we must always bear in mind. We shall discuss other examples and further ramifications in Chapter 4.

Strings are actually pointers to chars. In some cases, this realization can lead to compact code for string-processing functions. For example, to copy one string to another, we could write

```
while (*a++ = *b++) ;
```

instead of

```
for (i = 0; a[i] != 0; i++) a[i] = b[i];
```

or the third option given in Table 3.2. These two ways of referring to strings are equivalent, but may lead to code with different performance properties on different machines. We generally use the array version for clarity and the pointer version for economy, reserving detailed study of which is best for particular pieces of frequently executed code in particular applications.

Memory allocation for strings is more difficult than for linked lists because strings vary in size. Indeed, a fully general mechanism to reserve space for strings is neither more nor less than the system-provided `malloc` and `free` functions. As mentioned in Section 3.6, various algorithms have been developed for this problem, whose performance characteristics are system and machine dependent. Often, memory allocation is a less severe problem when we are working with strings than it might first appear, because we work with *pointers* to the strings, rather that with the characters themselves. Indeed, we *do not* normally assume in C code that all strings sit in individually allocated chunks of memory. We tend to assume that each string sits in memory of indeterminate allocation, just big enough for the string and its termination character. We must be very careful to ensure adequate allocation when we are performing operations that build or lengthen strings. As an example, we shall consider a program that reads strings and manipulates them in Section 3.7.

Exercises

▷ 3.56 Write a program that takes a string as argument, and that prints out a table giving, for each character that occurs in the string, the character and its frequency of occurrence.

▷3.57 Write a program that checks whether a given string is a palindrome (reads the same backward or forward), ignoring blanks. For example, your program should report success for the string `if i had a hifi`.

3.58 Suppose that memory for strings is individually allocated. Write versions of `strcpy` and `strcat` that allocate memory and return a pointer to the new string for the result.

3.59 Write a program that takes a string as argument and reads a sequence of words (sequences of characters separated by blank space) from standard input, printing out those that appear as substrings somewhere in the argument string.

3.60 Write a program that replaces substrings of more than one blank in a given string by exactly one blank.

3.61 Implement a pointer version of Program 3.15.

○3.62 Write an efficient program that finds the length of the longest sequence of blanks in a given string, examining as few characters in the string as possible. *Hint*: Your program should become faster as the length of the sequence of blanks increases.

3.7 Compound Data Structures

Arrays, linked lists, and strings all provide simple ways to structure data sequentially. They provide a first level of abstraction that we can use to group objects in ways amenable to processing the objects efficiently. Having settled on these abstractions, we can use them in a hierarchical fashion to build up more complex structures. We can contemplate arrays of arrays, arrays of lists, arrays of strings, and so forth. In this section, we consider examples of such structures.

In the same way that one-dimensional arrays correspond to vectors, *two-dimensional* arrays, with two indices, correspond to *matrices*, and are widely used in mathematical computations. For example, we might use the following code to multiply two matrices `a` and `b`, leaving the result in a third matrix `c`.

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0, c[i][j] = 0.0; k < N; k++)
      c[i][j] += a[i][k]*b[k][j];
```

We frequently encounter mathematical computations that are naturally expressed in terms of multidimensional arrays.

Program 3.16 Two-dimensional array allocation

This function dynamically allocates the memory for a two-dimensional array, as an array of arrays. We first allocate an array of pointers, then allocate memory for each row. With this function, the statement

```
int **a = malloc2d(M, N);
```

allocates an M-by-N array of integers.

```
int **malloc2d(int r, int c)
  { int i;
    int **t = malloc(r * sizeof(int *));
    for (i = 0; i < r; i++)
      t[i] = malloc(c * sizeof(int));
    return t;
  }
```

Beyond mathematical applications, a familiar way to structure information is to use a table of numbers organized into rows and columns. A table of students' grades in a course might have one row for each student, and one column for each assignment. In C, such a table would be represented as a two-dimensional array with one index for the row and one for the column. If we were to have 100 students and 10 assignments, we would write `grades[100][10]` to declare the array, and then refer to the ith student's grade on the jth assignment as `grade[i][j]`. To compute the average grade on an assignment, we sum together the elements in a column and divide by the number of rows; to compute a particular student's average grade in the course, we sum together the elements in a row and divide by the number of columns, and so forth. Two-dimensional arrays are widely used in applications of this type. On a computer, it is often convenient and straightforward to use more than two dimensions: An instructor might use a third index to keep student-grade tables for a sequence of years.

Two-dimensional arrays are a notational convenience, as the numbers are ultimately stored in the computer memory, which is essentially a one-dimensional array. In many programming environments, two-dimensional arrays are stored in *row-major order* in a one-dimensional array: In an array `a[M][N]`, the first N positions would be occupied by the first row (elements `a[0][0]` through `a[0][N-1]`),

Program 3.17 Sorting an array of strings

This program illustrates an an important string-processing function: rearranging a set of strings into sorted order. We read strings into a buffer large enough to hold them all, maintaining a pointer to each string in an array, then rearrange the pointers to put the pointer to the smallest string in the first position in the array, the pointer to the second smallest string in the second position in the array, and so forth.

The `qsort` library function that actually does the sort takes four arguments: a pointer to the beginning of the array, the number of objects, the size of each object, and a comparison function. It achieves independence from the type of object being sorted by blindly rearranging the blocks of data that represent objects (in this case string pointers) and by using a comparison function that takes pointers to `void` as argument. This code casts these back to type pointer to pointer to char for `strcmp`. To actually access the first character in a string for a comparison, we dereference three pointers: one to get the index (which is a pointer) into our array, one to get the pointer to the string (using the index), and one to get the character (using the pointer).

We use a different method to achieve type independence for our sorting and searching functions (see Chapters 4 and 6).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define Nmax 1000
#define Mmax 10000
char buf[Mmax]; int M = 0;
int compare(void *i, void *j)
  { return strcmp(*(char **)i, *(char **)j); }
main()
  { int i, N;
    char* a[Nmax];
    for (N = 0; N < Nmax; N++)
      {
        a[N] = &buf[M];
        if (scanf("%s", a[N]) == EOF) break;
        M += strlen(a[N])+1;
      }
    qsort(a, N, sizeof(char*), compare);
    for (i = 0; i < N; i++) printf("%s\n", a[i]);
  }
```

the second N positions by the second row (elements `a[1][0]` through `a[1][N-1]`), and so forth. With row-major order, the final line in the matrix-multiplication code in the beginning of this section is precisely equivalent to

```
c[N*i+j] = a[N*i+k]*b[N*k+j]
```

The same scheme generalizes to provide a facility for arrays with more dimensions. In C, multidimensional arrays may be implemented in a more general manner: we can define them to be compound data structures (arrays of arrays). This provides the flexibility, for example, to have an array of arrays that differ in size.

We saw a method in Program 3.6 for dynamic allocation of arrays that allows us to use our programs for varying problem sizes without recompiling them, and would like to have a similar method for multidimensional arrays. How do we allocate memory for multidimensional arrays whose size we do not know at compile time? That is, we want to be able to refer to an array element such as `a[i][j]` in a program, but cannot declare it as `int a[M][N]` (for example) because we do not know the values of `M` and `N`. For row-major order, a statement like

```
int* a = malloc(M*N*sizeof(int));
```

would be an effective way to allocate an M-by-N array of integers, but this solution will not work in all C environments, because not all implementations use row-major order. Program 3.16 gives a solution for two-dimensional arrays, based on their definition as arrays of arrays.

Program 3.17 illustrates the use of a similar compound structure: an array of strings. At first blush, since our abstract notion of a string is an array of characters, we might represent arrays of strings as arrays of arrays. However, the concrete representation that we use for a string in C is a *pointer* to the beginning of an array of characters, so an array of strings can also be an array of pointers. As illustrated in Figure 3.12, we then can get the effect of rearranging strings simply by rearranging the pointers in the array. Program 3.17 uses the `qsort` library function—implementing such functions is the subject of Chapters 6 through 9 in general and of Chapter 7 in particular. This example illustrates a typical scenario for processing strings: we read the characters themselves into a huge one-dimensional array, save

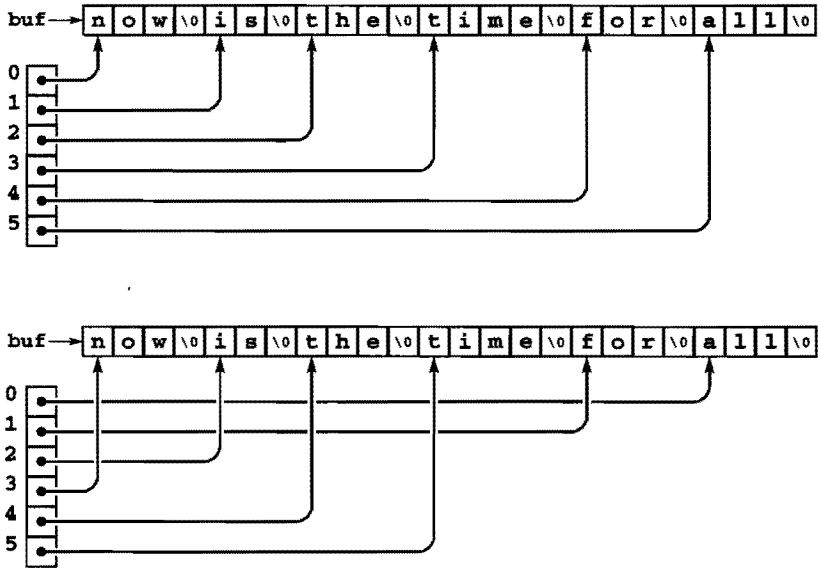


Figure 3.12
String sort

When processing strings, we normally work with pointers into a buffer that contains the strings (top), *because the pointers are easier to manipulate than the strings themselves, which vary in length. For example, the result of a sort is to rearrange the pointers such that accessing them in order gives the strings in alphabetical (lexicographic) order.*

pointers to individual strings (delimiting them with string-termination characters), then manipulate the pointers.

We have already encountered another use of arrays of strings: the `argv` array that is used to pass argument strings to `main` in C programs. The system stores in a string buffer the command line typed by the user and passes to `main` a pointer to an array of pointers to strings in that buffer. We use conversion functions to calculate numbers corresponding to some arguments; we use other arguments as strings, directly.

We can build compound data structures exclusively with links, as well. Figure 3.13 shows an example of a *multilist*, where nodes have multiple link fields and belong to independently maintained linked lists. In algorithm design, we often use more than one link to build up complex data structures, but in such a way that they are used to allow us to process them efficiently. For example, a doubly linked list is a multilist that satisfies the constraint that `x->l->r` and `x->r->l` are both equal to `x`. We shall examine a much more important data structure with two links per node in Chapter 5.

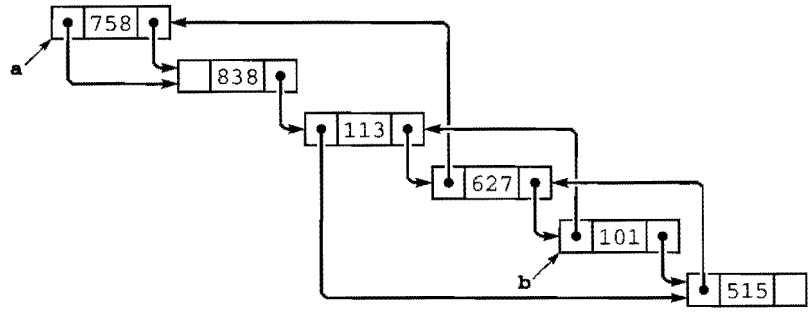


Figure 3.13
A multilist

We can link together nodes with two link fields in two independent lists, one using one link field, the other using the other link field. Here, the right link field links together nodes in one order (for example, this order could be the order in which the nodes were created) and the left link field links together nodes in a different order (for example, in this case, sorted order, perhaps the result of insertion sort using the left link field only). Following right links from a, *we visit the nodes in the order created; following left links from* b, *we visit the nodes in sorted order.*

If a multidimensional matrix is *sparse* (relatively few of the entries are nonzero), then we might use a multilist rather than a multidimensional array to represent it. We could use one node for each value in the matrix and one link for each dimension, with the link pointing to the next item in that dimension. This arrangement reduces the storage required from the product of the maximum indices in the dimensions to be proportional to the number of nonzero entries, but increases the time required for many algorithms, because they have to traverse links to access individual elements.

To see more examples of compound data structures and to highlight the distinction between indexed and linked data structures, we next consider data structures for representing graphs. A *graph* is a fundamental combinatorial object that is defined simply as a set of objects (called *vertices*) and a set of connections among the vertices (called *edges*). We have already encountered graphs, in the connectivity problem of Chapter 1.

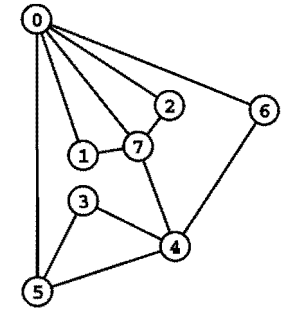
We assume that a graph with `V` vertices and `E` edges is defined by a set of `E` pairs of integers between `0` and `V-1`. That is, we assume that the vertices are labeled with the integers `0`, `1`, ..., `V-1`, and that the edges are specified as pairs of vertices. As in Chapter 1 we take the pair `i-j` as defining a connection between `i` and `j` and thus having the same meaning as the pair `j-i`. Graphs that comprise such edges are called *undirected* graphs. We shall consider other types of graphs in Part 7.

One straightforward method for representing a graph is to use a two-dimensional array, called an *adjacency matrix*. With an adjacency matrix, we can determine immediately whether or not there is an edge from vertex i to vertex j, just by checking whether row i and column

Program 3.18 Adjacency-matrix graph representation

This program reads a set of edges that define an undirected graph and builds an adjacency-matrix representation for the graph, setting `a[i][j]` and `a[j][i]` to 1 if there is an edge from `i` to `j` or `j` to `i` in the graph, or to 0 if there is no such edge. The program assumes that the number of vertices `V` is a compile-time constant. Otherwise, it would need to dynamically allocate the array that represents the adjacency matrix (see Exercise 3.72).

```
#include <stdio.h>
#include <stdlib.h>
main()
  { int i, j, adj[V][V];
    for (i = 0; i < V; i++)
      for (j = 0; j < V; j++)
        adj[i][j] = 0;
    for (i = 0; i < V; i++) adj[i][i] = 1;
    while (scanf("%d %d\n", &i, &j) == 2)
      { adj[i][j] = 1; adj[j][i] = 1; }
  }
```



	0	1	2	3	4	5	6	7
0	1	1	1	0	0	1	1	1
1	1	1	0	0	0	0	0	1
2	1	0	1	0	0	0	0	1
3	0	0	1	1	1	0	0	0
4	0	0	0	1	1	1	1	0
5	1	0	0	1	1	1	0	0
6	1	0	0	0	1	0	1	0
7	1	1	1	0	1	0	0	1

Figure 3.14
Graph with adjacency matrix representation

A graph is a set of vertices and a set of edges connecting the vertices. For simplicity, we assign indices (nonnegative integers, consecutively, starting at 0) to the vertices. An adjacency matrix is a two-dimensional array where we represent a graph by putting a 1 bit in row i and column j if and only if there is an edge from vertex i to vertex j. The array is symmetric about the diagonal. By convention, we assign 1 bits on the diagonal (each vertex is connected to itself). For example, the sixth row (and the sixth column) says that vertex 6 is connected to vertices 0, 4, and 6.

j of the matrix is nonzero. For the undirected graphs that we are considering, if there is an entry in row i and column j, then there also must be an entry in row j and column i, so the matrix is symmetric. Figure 3.14 shows an example of an adjacency matrix for an undirected graph; Program 3.18 shows how we can create an adjacency matrix, given a sequence of edges as input.

Another straightforward method for representing a graph is to use an array of linked lists, called *adjacency lists*. We keep a linked list for each vertex, with a node for each vertex connected to that vertex. For the undirected graphs that we are considering, if there is a node for j in i's list, then there must be a node for i in j's list. Figure 3.15 shows an example of the adjacency-lists representation of an undirected graph; Program 3.19 shows how we can create an adjacency-lists representation of a graph, given a sequence of edges as input.

Both graph representations are arrays of simpler data structures—one for each vertex describing the edges incident on that vertex. For

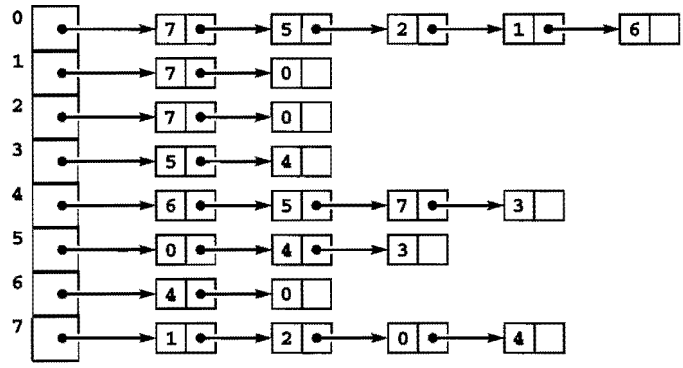


Figure 3.15
Adjacency-lists representation of a graph

This representation of the graph in Figure 3.14 uses an array of lists. The space required is proportional to the number of nodes plus the number of edges. To find the indices of the vertices connected to a given vertex i, we look at the ith position in an array, which contains a pointer to a linked list containing one node for each vertex connected to i.

an adjacency matrix, the simpler data structure is implemented as an indexed array; for an adjacency list, it is implemented as a linked list.

Thus, we face straightforward space tradeoffs when we represent a graph. The adjacency matrix uses space proportional to V^2; the adjacency lists use space proportional to $V + E$. If there are few edges (such a graph is said to be *sparse*), then the adjacency-lists representation uses far less space; if most pairs of vertices are connected by edges (such a graph is said to be *dense*), the adjacency-matrix representation might be preferable, because it involves no links. Some algorithms will be more efficient with the adjacency-matrix representation, because it allows the question "is there an edge between vertex i and vertex j?" to be answered in constant time; other algorithms will be more efficient with the adjacency-lists representation, because it allows us to process all the edges in a graph in time proportional to $V + E$, rather than to V^2. We see a specific example of this tradeoff in Section 5.8.

Both the adjacency-matrix and the adjacency-lists graph representations can be extended straightforwardly to handle other types of graphs (see, for example, Exercise 3.71). They serve as the basis for most of the graph-processing algorithms that we shall consider in Part 7.

To conclude this chapter, we consider an example that shows the use of compound data structures to provide an efficient solution to the simple geometric problem that we considered in Section 3.2. Given d, we want to know how many pairs from a set of N points in the unit square can be connected by a straight line of length less than d.

Program 3.19 Adjacency-lists graph representation

This program reads a set of edges that define a graph and builds an adjacency-matrix representation for the graph. An adjacency list for a graph is an array of lists, one for each vertex, where the jth list contains a linked list of the nodes connected to the jth vertex.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct node *link;
struct node
  { int v; link next; };
link NEW(int v, link next)
  { link x = malloc(sizeof *x);
    x->v = v; x->next = next;
    return x;
  }
main()
  { int i, j; link adj[V];
    for (i = 0; i < V; i++) adj[i] = NULL;
    while (scanf("%d %d\n", &i, &j) == 2)
      {
        adj[j] = NEW(i, adj[j]);
        adj[i] = NEW(j, adj[i]);
      }
  }
```

Program 3.20 uses a two-dimensional array of linked lists to improve the running time of Program 3.8 by a factor of about $1/d^2$ when N is sufficiently large. It divides the unit square up into a grid of equal-sized smaller squares. Then, for each square, it builds a linked list of all the points that fall into that square. The two-dimensional array provides the capability to access immediately the set of points close to a given point; the linked lists provide the flexibility to store the points where they may fall without our having to know ahead of time how many points fall into each grid square.

The space used by Program 3.20 is proportional to $1/d^2 + N$, but the running time is $O(d^2N^2)$, which is a substantial improvement over the brute-force algorithm of Program 3.8 for small d. For exam-

Program 3.20 A two-dimensional array of lists

This program illustrates the effectiveness of proper data-structure choice, for the geometric computation of Program 3.8. It divides the unit square into a grid, and maintains a two-dimensional array of linked lists, with one list corresponding to each grid square. The grid is chosen to be sufficiently fine that all points within distance d of any given point are either in the same grid square or an adjacent one. The function `malloc2d` is like the one in Program 3.16, but for objects of type `link` instead of `int`.

```
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include "Point.h"
typedef struct node* link;
struct node { point p;  link next; };
link **grid; int G; float d; int cnt = 0;
gridinsert(float x, float y)
  { int i, j; link s;
    int X = x*G +1; int Y = y*G+1;
    link t = malloc(sizeof *t);
    t->p.x = x; t->p.y = y;
    for (i = X-1; i <= X+1; i++)
      for (j = Y-1; j <= Y+1; j++)
        for (s = grid[i][j]; s != NULL; s = s->next)
          if (distance(s->p, t->p) < d) cnt++;
    t->next = grid[X][Y]; grid[X][Y] = t;
  }
main(int argc, char *argv[])
 { int i, j, N = atoi(argv[1]);
   d = atof(argv[2]); G = 1/d;
   grid = malloc2d(G+2, G+2);
   for (i = 0; i < G+2; i++)
     for (j = 0; j < G+2; j++)
       grid[i][j] = NULL;
   for (i = 0; i < N; i++)
     gridinsert(randFloat(), randFloat());
   printf("%d edges shorter than %f\n", cnt, d);
 }
```

ple, with $N = 10^6$ and $d = 0.001$, we can solve the problem in time and space that is effectively linear, whereas the brute-force algorithm would require a prohibitive amount of time. We can use this data structure as the basis for solving many other geometric problems, as well. For example, combined with a union-find algorithm from Chapter 1, it gives a near-linear algorithm for determining whether a set of N random points in the plane can be connected together with lines of length d—a fundamental problem of interest in networking and circuit design.

As suggested by the examples that we have seen in this section, there is no end to the level of complexity that we can build up from the basic abstract constructs that we can use to structure data of differing types into objects and sequence the objects into compound objects, either implicitly or with explicit links. These examples still leave us one step away from full generality in structuring data, as we shall see in Chapter 5. Before taking that step, however, we shall consider the important abstract data structures that we can build with linked lists and arrays—basic tools that will help us in developing the next level of generality.

Exercises

3.63 Write a version of Program 3.16 that handles *three*-dimensional arrays.

3.64 Modify Program 3.17 to process input strings individually (allocate memory for each string after reading it from the input). You can assume that all strings have less than 100 characters.

3.65 Write a program to fill in a two-dimensional array of 0–1 values by setting `a[i][j]` to 1 if the greatest common divisor of `i` and `j` is 1, and to 0 otherwise.

3.66 Use Program 3.20 in conjunction with Program 1.4 to develop an efficient program that can determine whether a set of N points can be connected with edges of length less than d.

3.67 Write a program to convert a sparse matrix from a two-dimensional array to a multilist with nodes for only nonzero values.

• 3.68 Implement matrix multiplication for matrices represented with multilists.

▷ 3.69 Show the adjacency matrix that is built by Program 3.18 given the input pairs `0-2`, `1-4`, `2-5`, `3-6`, `0-4`, `6-0`, and `1-3`.

▷ 3.70 Show the adjacency lists that are built by Program 3.19 given the input pairs `0-2`, `1-4`, `2-5`, `3-6`, `0-4`, `6-0`, and `1-3`.

o 3.71 A *directed* graph is one where vertex connections have orientations: edges go *from* one vertex *to* another. Do Exercises 3.69 and 3.70 under the assumption that the input pairs represent a directed graph, with `i-j` signifying that there is an edge from `i` to `j`. Also, draw the graph, using arrows to indicate edge orientations.

3.72 Modify Program 3.18 to take the number of vertices as a command-line argument, then dynamically allocate the adjacency matrix.

3.73 Modify Program 3.19 to take the number of vertices as a command-line argument, then dynamically allocate the array of lists.

o 3.74 Write a function that uses the adjacency matrix of a graph to calculate, given vertices a and b, the number of vertices c with the property that there is an edge from a to c and from c to b.

o 3.75 Answer Exercise 3.74, but use adjacency lists.

CHAPTER FOUR

Abstract Data Types

DEVELOPING ABSTRACT MODELS for our data and for the ways in which our programs process those data is an essential ingredient in the process of solving problems with a computer. We see examples of this principle at a low level in everyday programming (for example when we use arrays and linked lists, as discussed in Chapter 3) and at a high level in problem-solving (as we saw in Chapter 1, when we used union–find forests to solve the connectivity problem). In this chapter, we consider *abstract data types (ADTs)*, which allow us to build programs that use high-level abstractions. With abstract data types, we can separate the conceptual transformations that our programs perform on our data from any particular data-structure representation and algorithm implementation.

All computer systems are based on *layers of abstraction*: We adopt the abstract model of a bit that can take on a binary 0–1 value from certain physical properties of silicon and other materials; then, we adopt the abstract model of a machine from dynamic properties of the values of a certain set of bits; then, we adopt the abstract model of a programming language that we realize by controlling the machine with a machine-language program; then, we adopt the abstract notion of an algorithm implemented as a C language program. Abstract data types allow us to take this process further, to develop abstract mechanisms for certain computational tasks at a higher level than provided by the C system, to develop application-specific abstract mechanisms that are suitable for solving problems in numerous applications areas, and to build higher-level abstract mechanisms that use these basic

mechanisms. Abstract data types give us an ever-expanding set of tools that we can use to attack new problems.

On the one hand, our use of abstract mechanisms frees us from detailed concern about how they are implemented; on the other hand, when performance matters in a program, we need to be cognizant of the costs of basic operations. We use many basic abstractions that are built into the computer hardware and provide the basis for machine instructions; we implement others in software; and we use still others that are provided in previously written systems software. Often, we build higher-level abstract mechanisms in terms of more primitive ones. The same basic principle holds at all levels: We want to identify the critical operations in our programs and the critical characteristics of our data, to define both precisely at an abstract level, and to develop efficient concrete mechanisms to support them. We consider many examples of this principle in this chapter.

To develop a new layer of abstraction, we need to *define* the abstract objects that we want to manipulate and the operations that we perform on them; we need to *represent* the data in some data structure and to *implement* the operations; and (the point of the exercise) we want to ensure that the objects are convenient to *use* to solve an applications problem. These comments apply to simple data types as well, and the basic mechanisms that we discussed in Chapter 3 to support data types will serve our purposes, with one significant extension.

Definition 4.1 *An* **abstract data type (ADT)** *is a data type (a set of values and a collection of operations on those values) that is accessed only through an* **interface**. *We refer to a program that uses an ADT as a* **client**, *and a program that specifies the data type as an* **implementation**.

The key distinction that makes a data type abstract is drawn by the word *only*: with an ADT, client programs do not access any data values except through the operations provided in the interface. The representation of the data and the functions that implement the operations are in the implementation, and are completely separated from the client, by the interface. We say that the interface is *opaque*: the client cannot see the implementation through the interface.

For example, the interface for the data type for points (Program 3.3) in Section 3.1 explicitly declares that points are represented

as structures with pairs of floats, with members named x and y. Indeed, this use of data types is common in large software systems: we develop a set of conventions for how data is to be represented (and define a number of associated operations) and make those conventions available in an interface for use by client programs that comprise a large system. The data type ensures that all parts of the system are in agreement on the representation of core system-wide data structures. While valuable, this strategy has a flaw: if we need to *change* the data representation, then we need to change all the client programs. Program 3.3 again provides a simple example: one reason for developing the data type is to make it convenient for client programs to manipulate points, and we expect that clients will access the individual coordinates when needed. But we cannot change to a different representation (polar coordinates, say, or three dimensions, or even different data types for the individual coordinates) without changing all the client programs.

Our implementation of a simple list-processing interface in Section 3.4 (Program 3.12) is an example of a first step towards an ADT. In the client program that we considered (Program 3.13), we adopted the convention that we would access the data only through the operations defined in the interface, and were therefore able to consider changing the representation without changing the client (see Exercise 3.52). Adopting such a convention amounts to using the data type as though it was abstract, but leaves us exposed to subtle bugs, because the data representation remains available to clients, in the interface, and we would have to be vigilant to ensure that they do not depend upon it, even if accidentally. With true ADTs, we provide no information to clients about data representation, and are thus free to change it.

Definition 4.1 does not specify what an interface is or how the data type and the operations are to be described. This imprecision is necessary because specifying such information in full generality requires a formal mathematical language and eventually leads to difficult mathematical questions. This question is central in programming language design. We shall discuss the specification problem further after we consider examples of ADTs.

ADTs have emerged as an effective mechanism for organizing large modern software systems. They provide a way to limit the size and complexity of the interface between (potentially complicated) al-

gorithms and associated data structures and (a potentially large number of) programs that use the algorithms and data structures. This arrangement makes it easier to understand a large applications program as a whole. Moreover, unlike simple data types, ADTs provide the flexibility necessary to make it convenient to change or improve the fundamental data structures and algorithms in the system. Most important, the ADT interface defines a contract between users and implementors that provides a precise means of communicating what each can expect of the other.

We examine ADTs in detail in this chapter because they also play an important role in the study of data structures and algorithms. Indeed, the essential motivation behind the development of nearly all the algorithms that we consider in this book is to provide efficient implementations of the basic operations for certain fundamental ADTs that play a critical role in many computational tasks. Designing an ADT is only the first step in meeting the needs of applications programs—we also need to develop viable implementations of the associated operations and underlying data structures that enable them. Those tasks are the topic of this book. Moreover, we use abstract models directly to develop and to compare the performance characteristics of algorithms and data structures, as in the example in Chapter 1: Typically, we develop an applications program that uses an ADT to solve a problem, then develop multiple implementations of the ADT and compare their effectiveness. In this chapter, we consider this general process in detail, with numerous examples.

C programmers use data types and ADTs regularly. At a low level, when we process integers using only the operations provided by C for integers, we are essentially using a system-defined abstraction for integers. The integers could be represented and the operations implemented some other way on some new machine, but a program that uses only the operations specified for integers will work properly on the new machine. In this case, the various C operations for integers constitute the interface, our programs are the clients, and the system hardware and software provide the implementation. Often, the data types are sufficiently abstract that we can move to a new machine with, say, different representations for integers or floating point numbers, without having to change programs (though this ideal is not achieved as often as we would like).

At a higher level, as we have seen, C programmers often define interfaces in the form of .h files that describe a set of operations on some data structure, with implementations in some independent .c file. This arrangement provides a contract between user and implementor, and is the basis for the standard libraries that are found in C programming environments. However, many such libraries comprise operations on a particular data structure, and therefore constitute data types, but not *abstract* data types. For example, the C string library is not an ADT because programs that use strings know how strings are represented (arrays of characters) and typically access them directly via array indexing or pointer arithmetic. We could not switch, for example, to a linked-list representation of strings without changing the client programs. The memory-allocation interface and implementation for linked lists that we considered in Sections 3.4 and 3.5 has this same property. By contrast, ADTs allow us to develop implementations that not only use different implementations of the operations, but also involve different underlying data structures. Again, the key distinction that characterizes ADTs is the requirement that the data type be accessed *only* through the interface.

We shall see many examples of data types that *are* abstract throughout this chapter. After we have developed a feel for the concept, we shall return to a discussion of philosophical and practical implications, at the end of the chapter.

4.1 Abstract Objects and Collections of Objects

The data structures that we use in applications often contain a great deal of information of various types, and certain pieces of information may belong to multiple independent data structures. For example, a file of personnel data may contain records with names, addresses, and various other pieces of information about employees; and each record may need to belong to one data structure for searching for particular employees, to another data structure for answering statistical queries, and so forth.

Despite this diversity and complexity, a large class of computing applications involve generic manipulation of data objects, and need access to the information associated with them for a limited number of specific reasons. Many of the manipulations that are required are

a natural outgrowth of basic computational procedures, so they are needed in a broad variety of applications. Many of the fundamental algorithms that we consider in this book can be applied effectively to the task of building a layer of abstraction that can provide client programs with the ability to perform such manipulations efficiently. Thus, we shall consider in detail numerous ADTs that are associated with such manipulations. They define various operations on collections of abstract objects, independent of the type of the object.

We have discussed the use of simple data types in order to write code that does not depend on object types, in Chapter 3, where we used `typedef` to specify the type of our data items. This approach allows us to use the same code for, say, integers and floating-point numbers, just by changing the `typedef`. With pointers, the object types can be arbitrarily complex. When we use this approach, we are making implicit assumptions about the operations that we perform on the objects, and we are not hiding the data representation from our client programs. ADTs provide a way for us to make explicit any assumptions about the operations that we perform on data objects.

We will consider a general mechanism for the purpose of building ADTs for generic data objects in detail in Section 4.8. It is based on having the interface defined in a file named `Item.h`, which provides us with the ability to declare variables of type `Item`, and to use these variables in assignment statements, as function arguments, and as function return values. In the interface, we explicitly define any operations that our algorithms need to perform on generic objects. The mechanism that we shall consider allows us to do all this without providing any information about the data representation to client programs, thus giving us a true ADT.

For many applications, however, the different types of generic objects that we want to consider are simple and similar, and it is essential that the implementations be as efficient as possible, so we often use simple data types, not true ADTs. Specifically, we often use `Item.h` files that describe the objects themselves, not an interface. Most often, this description consists of a `typedef` to define the data type and a few macros to define the operations. For example, for an application where the only operation that we perform on the data (beyond the generic ones enabled by the `typedef`) is `eq` (test whether

two items are the same), we would use an `Item.h` file comprising the two lines of code:

```
typedef int Item
#define eq(A, B) (A == B) .
```

Any client program with the line `#include Item.h` can use `eq` to test whether two items are equal (as well as using items in declarations, assignment statements, and function arguments and return values) in the code implementing some algorithm. Then we could use that same client program for strings, for example, by changing `Item.h` to

```
typedef char* Item;
#define eq(A, B) (strcmp(A, B) == 0) .
```

This arrangement does not constitute the use of an ADT because the particular data representation is freely available to any program that includes `Item.h`. We typically would add macros or function calls for other simple operations on items (for example to print them, read them, or set them to random values). We adopt the convention in our client programs that we use items *as though* they were defined in an ADT, to allow us to leave the types of our basic objects unspecified in our code without any performance penalty. To use a true ADT for such a purpose would be overkill for many applications, but we shall discuss the possibility of doing so in Section 4.8, after we have seen many other examples. In principle, we can apply the technique for arbitrarily complicated data types, although the more complicated the type, the more likely we are to consider the use of a true ADT.

Having settled on some method for implementing data types for generic objects, we can move on to consider *collections* of objects. Many of the data structures and algorithms that we consider in this book are used to implement fundamental ADTs comprising collections of abstract objects, built up from the following two operations:

- *insert* a new object into the collection.
- *delete* an object from the collection.

We refer to such ADTs as *generalized queues*. For convenience, we also typically include explicit operations to *initialize* the data structure and to *count* the number of items in the data structure (or just to test whether it is empty). Alternatively, we could encompass these operations within *insert* and *delete* by defining appropriate return values.

We also might wish to *destroy* the data structure or to *copy* it; we shall discuss such operations in Section 4.8.

When we *insert* an object, our intent is clear, but which object do we get when we *delete* an object from the collection? Different ADTs for collections of objects are characterized by different criteria for deciding which object to remove for the *delete* operation and by different conventions associated with the various criteria. Moreover, we shall encounter a number of other natural operations beyond *insert* and *delete*. Many of the algorithms and data structures that we consider in this book were designed to support efficient implementation of various subsets of these operations, for various different *delete* criteria and other conventions. These ADTs are conceptually simple, used widely, and lie at the core of a great many computational tasks, so they deserve the careful attention that we pay them.

We consider several of these fundamental data structures, their properties, and examples of their application while at the same time using them as examples to illustrate the basic mechanisms that we use to develop ADTs. In Section 4.2, we consider the *pushdown stack*, where the rule for removing an object is to remove the one that was most recently inserted. We consider applications of stacks in Section 4.3, and implementations in Section 4.4, including a specific approach to keeping the applications and implementations separate. Following our discussion of stacks, we step back to consider the process of creating a new ADT, in the context of the union–find abstraction for the connectivity problem that we considered in Chapter 1. Following that, we return to collections of abstract objects, to consider FIFO queues and generalized queues (which differ from stacks on the abstract level only in that they involve using a different rule to remove items) and generalized queues where we disallow duplicate items.

As we saw in Chapter 3, arrays and linked lists provide basic mechanisms that allow us to *insert* and *delete* specified items. Indeed, linked lists and arrays are the underlying data structures for several of the implementations of generalized queues that we consider. As we know, the cost of insertion and deletion is dependent on the specific structure that we use and the specific item being inserted or deleted. For a given ADT, our challenge is to choose a data structure that allows us to perform the required operations efficiently. In this chapter, we examine in detail several examples of ADTs for which linked lists and

arrays provide appropriate solutions. ADTs that support more powerful operations require more sophisticated implementations, which are the prime impetus for many of the algorithms that we consider in this book.

Data types comprising collections of abstract objects (generalized queues) are a central object of study in computer science because they directly support a fundamental paradigm of computation. For a great many computations, we find ourselves in the position of having many objects with which to work, but being able to process only one object at a time. Therefore, we need to save the others while processing that one. This processing might involve examining some of the objects already saved away or adding more to the collection, but operations of saving the objects away and retrieving them according to some criterion are the basis of the computation. Many classical data structures and algorithms fit this mold, as we shall see.

Exercises

▷4.1 Give a definition for `Item` and `eq` that might be used for floating-point numbers, where two floating-point numbers are considered to be equal if the absolute value of their difference divided by the larger (in absolute value) of the two numbers is less than 10^{-6}.

▷4.2 Give a definition for `Item` and `eq` that might be used for points in the plane (see Section 3.1).

4.3 Add a macro `ITEMshow` to the generic object type definitions for integers and strings described in the text. Your macro should print the value of the item on standard output.

▷4.4 Give definitions for `Item` and `ITEMshow` (see Exercise 4.3) that might be used in programs that process playing cards.

4.5 Rewrite Program 3.1 to use a generic object type in a file `Item.h`. Your object type should include `ITEMshow` (see Exercise 4.3) and `ITEMrand`, so that the program can be used for any type of number for which + and / are defined.

4.2 Pushdown Stack ADT

Of the data types that support *insert* and *delete* for collections of objects, the most important is called the *pushdown stack*.

A stack operates somewhat like a busy professor's "in" box: work piles up in a stack, and whenever the professor has a chance to get some work done, it comes off the top. A student's paper might

well get stuck at the bottom of the stack for a day or two, but a conscientious professor might manage to get the stack emptied at the end of the week. As we shall see, computer programs are naturally organized in this way. They frequently postpone some tasks while doing others; moreover, they frequently need to return to the most recently postponed task first. Thus, pushdown stacks appear as the fundamental data structure for many algorithms.

L		L
A		L A
*	A	L
S		L S
T		L S T
I		L S T I
*	I	L S T
N		L S T N
*	N	L S T
F		L S T F
I		L S T F I
R		L S T F I R
*	R	L S T F I
S		L S T F I S
T		L S T F I S T
*	T	L S T F I S
*	S	L S T F I
O		L S T F I O
U		L S T F I O U
*	U	L S T F I O
T		L S T F I O T
*	T	L S T F I O
*	O	L S T F I
*	I	L S T F
*	F	L S T
*	T	L S
*	S	L
*	L	

Figure 4.1
Pushdown stack (LIFO queue) example

This list shows the result of the sequence of operations in the left column (top to bottom), where a letter denotes push *and an asterisk denotes* pop. *Each line displays the operation, the letter popped for pop operations, and the contents of the stack after the operation, in order from least recently inserted to most recently inserted, left to right.*

Definition 4.2 *A* **pushdown stack** *is an ADT that comprises two basic operations: insert (***push***) a new item, and delete (***pop***) the item that was most recently inserted.*

That is, when we speak of a *pushdown stack ADT*, we are referring to a description of the *push* and *pop* operations that is sufficiently well specified that a client program can make use of them, and to some implementation of the operations enforcing the rule that characterizes a pushdown stack: items are removed according to a *last-in, first-out (LIFO)* discipline. In the simplest case, which we use most often, both client and implementation refer to just a single stack (that is, the "set of values" in the data type is just that one stack); in Section 4.8, we shall see how to build an ADT that supports multiple stacks.

Figure 4.1 shows how a sample stack evolves through a series of *push* and *pop* operations. Each *push* increases the size of the stack by 1 and each *pop* decreases the size of the stack by 1. In the figure, the items in the stack are listed in the order that they are put on the stack, so that it is clear that the rightmost item in the list is the one at the top of the stack—the item that is to be returned if the next operation is *pop*. In an implementation, we are free to organize the items any way that we want, as long as we allow clients to maintain the illusion that the items are organized in this way.

To write programs that use the pushdown stack abstraction, we need first to define the interface. In C, one way to do so is to declare the four operations that client programs may use, as illustrated in Program 4.1. We keep these declarations in a file `STACK.h` that is referenced as an include file in client programs and implementations.

Furthermore, we expect that there is no other connection between client programs and implementations. We have already seen, in Chapter 1, the value of identifying the abstract operations on which a computation is based. We are now considering a mechanism that

Program 4.1 Pushdown-stack ADT interface

This interface defines the basic operations that define a pushdown stack. We assume that the four declarations here are in a file `STACK.h`, which is referenced as an include file by client programs that use these functions and implementations that provide their code; and that both clients and implementations define `Item`, perhaps by including an `Item.h` file (which may have a `typedef` or which may define a more general interface). The argument to `STACKinit` specifies the maximum number of elements expected on the stack.

```
void STACKinit(int);
 int STACKempty();
void STACKpush(Item);
Item STACKpop();
```

allows us to write programs that use these abstract operations. To enforce the abstraction, we hide the data structure and the implementation from the client. In Section 4.3, we consider examples of client programs that use the stack abstraction; in Section 4.4, we consider implementations.

In an ADT, the purpose of the interface is to serve as a contract between client and implementation. The function declarations ensure that the calls in the client program and the function definitions in the implementation match, but the interface otherwise contains no information about how the functions are to be implemented, or even how they are to behave. How can we explain what a stack is to a client program? For simple structures like stacks, one possibility is to exhibit the code, but this solution is clearly not effective in general. Most often, programmers resort to English-language descriptions, in documentation that accompanies the code.

A rigorous treatment of this situation requires a full description, in some formal mathematical notation, of how the functions are supposed to behave. Such a description is sometimes called a *specification*. Developing a specification is generally a challenging task. It has to describe *any* program that implements the functions in a mathematical metalanguage, whereas we are used to specifying the behavior of functions with code written in a programming language. In practice, we describe behavior in English-language descriptions. Before getting

drawn further into epistemological issues, we move on. In this book, we give detailed examples, English-language descriptions, and multiple implementations for most of the ADTs that we consider.

To emphasize that our specification of the pushdown stack ADT is sufficient information for us to write meaningful client programs, we consider, in Section 4.3, two client programs that use pushdown stacks, before considering any implementation.

Exercises

▷ **4.6** A letter means *push* and an asterisk means *pop* in the sequence

E A S * Y * Q U E * * * S T * * * I O * N * * *.

Give the sequence of values returned by the *pop* operations.

4.7 Using the conventions of Exercise 4.6, give a way to insert asterisks in the sequence E A S Y so that the sequence of values returned by the *pop* operations is (*i*) E A S Y ; (*ii*) Y S A E ; (*iii*) A S Y E ; (*iv*) A Y E S ; or, in each instance, prove that no such sequence exists.

•• **4.8** Given two sequences, give an algorithm for determining whether or not asterisks can be added to make the first produce the second, when interpreted as a sequence of stack operations in the sense of Exercise 4.7.

4.3 Examples of Stack ADT Clients

We shall see a great many applications of stacks in the chapters that follow. As an introductory example, we now consider the use of stacks for evaluating arithmetic expressions. For example, suppose that we need to find the value of a simple arithmetic expression involving multiplication and addition of integers, such as

```
5 * ( ( ( 9 + 8 ) * ( 4 * 6 ) ) + 7 )
```

The calculation involves saving intermediate results: For example, if we calculate `9 + 8` first, then we have to save the result `17` while, say, we compute `4 * 6`. A pushdown stack is the ideal mechanism for saving intermediate results in such a calculation.

We begin by considering a simpler problem, where the expression that we need to evaluate is in a form where each operator appears *after* its two arguments, rather than between them. As we shall see, any arithmetic expression can be arranged in this form, which is called

postfix, by contrast with *infix*, the customary way of writing arithmetic expressions. The postfix representation of the expression in the previous paragraph is

```
5 9 8 + 4 6 * * 7 + *
```

The reverse of postfix is called *prefix*, or *Polish notation* (because it was invented by the Polish logician Lukasiewicz).

In infix, we need parentheses to distinguish, for example,

```
5 * ( ( ( 9 + 8 ) * ( 4 * 6 ) ) + 7 )
```

from

```
( ( 5 * 9 ) + 8 ) * ( ( 4 * 6 ) + 7 )
```

but parentheses are unnecessary in postfix (or prefix). To see why, we can consider the following process for converting a postfix expression to an infix expression: We replace all occurrences of two operands followed by an operator by their infix equivalent, with parentheses, to indicate that the result can be considered to be an operand. That is, we replace any occurrence of `a b *` and `a b +` by `(a * b)` and `(a + b)`, respectively. Then, we perform the same transformation on the resulting expression, continuing until all the operators have been processed. For our example, the transformation happens as follows:

```
5 9 8 + 4 6 * * 7 + *
5 ( 9 + 8 ) ( 4 * 6 ) * 7 + *
5 ( ( 9 + 8 ) * ( 4 * 6 ) ) 7 + *
5 ( ( ( 9 + 8 ) * ( 4 * 6 ) ) + 7 ) *
( 5 * ( ( ( 9 + 8 ) * ( 4 * 6 ) ) + 7 ) )
```

We can determine the operands associated with any operator in the postfix expression in this way, so no parentheses are necessary.

Alternatively, with the aid of a stack, we can actually perform the operations and evaluate any postfix expression, as illustrated in Figure 4.2. Moving from left to right, we interpret each operand as the command to "push the operand onto the stack," and each operator as the commands to "pop the two operands from the stack, perform the operation, and push the result." Program 4.2 is a C implementation of this process.

Postfix notation and an associated pushdown stack give us a natural way to organize a series of computational procedures. Some calculators and some computing languages explicitly base their method

```
5       5
9       5   9
8       5   9   8
+       5  17
4       5  17   4
6       5  17   4   6
*       5  17  24
*       5 408
7       5 408   7
+       5 415
*    2075
```

Figure 4.2
Evaluation of a postfix expression

This sequence shows the use of a stack to evaluate the postfix expression `5 9 8 + 4 6 * * 7 + *`. *Proceeding from left to right through the expression, if we encounter a number, we push it on the stack; and if we encounter an operator, we push the result of applying the operator to the top two numbers on the stack.*

Program 4.2 Postfix-expression evaluation

This pushdown-stack client reads any postfix expression involving multiplication and addition of integers, then evaluates the expression and prints the computed result.

When we encounter operands, we push them on the stack; when we encounter operators, we pop the top two entries from the stack and push the result of applying the operator to them. The order in which the two `STACKpop()` operations are performed in the expressions in this code is unspecified in C, so the code for noncommutative operators such as subtraction or division would be slightly more complicated.

The program assumes that at least one blank follows each integer, but otherwise does not check the legality of the input at all. The final `if` statement and the `while` loop perform a calculation similar to the C `atoi` function, which converts integers from ASCII strings to integers for calculation. When we encounter a new digit, we multiply the accumulated result by 10 and add the digit.

The stack contains integers—that is, we assume that `Item` is defined to be `int` in `Item.h`, and that `Item.h` is also included in the stack implementation (see, for example, Program 4.4).

```
#include <stdio.h>
#include <string.h>
#include "Item.h"
#include "STACK.h"
main(int argc, char *argv[])
  { char *a = argv[1]; int i, N = strlen(a);
    STACKinit(N);
    for (i = 0; i < N; i++)
      {
        if (a[i] == '+')
          STACKpush(STACKpop()+STACKpop());
        if (a[i] == '*')
          STACKpush(STACKpop()*STACKpop());
        if ((a[i] >= '0') && (a[i] <= '9'))
          STACKpush(0);
        while ((a[i] >= '0') && (a[i] <= '9'))
          STACKpush(10*STACKpop() + (a[i++]-'0'));
      }
    printf("%d \n", STACKpop());
  }
```

of calculation on postfix and stack operations—every operation pops its arguments from the stack and returns its results to the stack.

One example of such a language is the PostScript language, which is used to print this book. It is a complete programming language where programs are written in postfix and are interpreted with the aid of an internal stack, precisely as in Program 4.2. Although we cannot cover all the aspects of the language here (*see reference section*), it is sufficiently simple that we can study some actual programs, to appreciate the utility of the postfix notation and the pushdown-stack abstraction. For example, the string

```
5 9 8 add 4 6 mul mul 7 add mul
```

is a PostScript program! Programs in PostScript consist of operators (such as `add` and `mul`) and operands (such as integers). As we did in Program 4.2 we interpret a program by reading it from left to right: If we encounter an operand, we push it onto the stack; if we encounter an operator, we pop its operands (if any) from the stack and push the result (if any). Thus, the execution of this program is fully described by Figure 4.2: The program leaves the value 2075 on the stack.

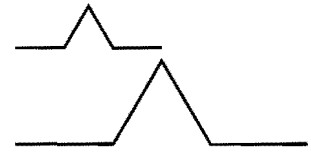
PostScript has a number of primitive functions that serve as instructions to an abstract plotting device; we can also define our own functions. These functions are invoked with arguments on the stack in the same way as any other function. For example, the PostScript code

```
0 0 moveto 144 hill 0 72 moveto 72 hill stroke
```

corresponds to the sequence of actions "call `moveto` with arguments `0` and `0`, then call `hill` with argument `144`," and so forth. Some operators refer directly to the stack itself. For example the operator `dup` duplicates the entry at the top of the stack so, for example, the PostScript code

```
144 dup 0 rlineto 60 rotate dup 0 rlineto
```

corresponds to the sequence of actions "call `rlineto` with arguments `144` and `0`, then call `rotate` with argument `60`, then call `rlineto` with arguments `144` and `0`," and so forth. The PostScript program in Figure 4.3 defines and uses the function `hill`. Functions in PostScript are like macros: The sequence `/hill { A } def` makes the name `hill` equivalent to the operator sequence inside the braces. Figure 4.3 is an example of a PostScript program that defines a function and draws a simple diagram.



```
/hill {
      dup 0 rlineto
      60 rotate
      dup 0 rlineto
      -120 rotate
      dup 0 rlineto
      60 rotate
      dup 0 rlineto
      pop
    } def
0 0 moveto
144 hill
0 72 moveto
72 hill
stroke
```

Figure 4.3
Sample PostScript program

The diagram at the top was drawn by the PostScript program below it. The program is a postfix expression that uses the built-in functions `moveto`, `rlineto`, `rotate`, `stroke` *and* `dup`; *and the user-defined function* `hill` (*see text*). *The graphics commands are instructions to a plotting device:* `moveto` *instructs that device to go to the specified position on the page (coordinates are in points, which are 1/72 inch);* `rlineto` *instructs it to move to the specified position in coordinates relative to its current position, adding the line it makes to its current path;* `rotate` *instructs it to turn left the specified number of degrees; and* `stroke` *instructs it to draw the path that it has traced.*

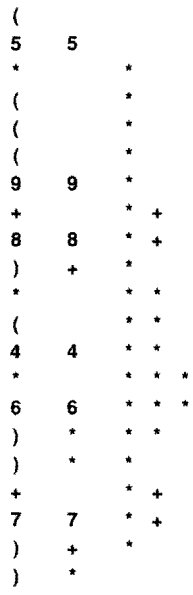


Figure 4.4
Conversion of an infix expression to postfix

This sequence shows the use of a stack to convert the infix expression `(5*(((9+8)*(4*6))+7))` *to its postfix form* `5 9 8 + 4 6 * * 7 + * .` *We proceed from left to right through the expression: If we encounter a number, we write it to the output; if we encounter a left parenthesis, we ignore it; if we encounter an operator, we push it on the stack; and if we encounter a right parenthesis, we write the operator at the top of the stack to the output.*

In the present context, our interest in PostScript is that this widely used programming language is based on the pushdown-stack abstraction. Indeed, many computers implement basic stack operations in hardware because they naturally implement a function-call mechanism: Save the current environment on entry to a function by pushing information onto a stack; restore the environment on exit by using information popped from the stack. As we see in Chapter 5, this connection between pushdown stacks and programs organized as functions that call functions is an essential paradigm of computation.

Returning to our original problem, we can also use a pushdown stack to convert fully parenthesized arithmetic expressions from infix to postfix, as illustrated in Figure 4.4. For this computation, we push the *operators* onto a stack, and simply pass the operands through to the output. Then, each right parenthesis indicates that both arguments for the last operator have been output, so the operator itself can be popped and output.

Program 4.3 is an implementation of this process. Note that arguments appear in the postfix expression in the same order as in the infix expression. It is also amusing to note that the left parentheses are not needed in the infix expression. The left parentheses would be required, however, if we could have operators that take differing numbers of operands (see Exercise 4.11).

In addition to providing two different examples of the use of the pushdown-stack abstraction, the entire algorithm that we have developed in this section for evaluating infix expressions is itself an exercise in abstraction. First, we convert the input to an intermediate representation (postfix). Second, we simulate the operation of an abstract stack-based machine to interpret and evaluate the expression. This same schema is followed by many modern programming-language translators, for efficiency and portability: The problem of compiling a C program for a particular computer is broken into two tasks centered around an intermediate representation, so that the problem of translating the program is separated from the problem of executing that program, just as we have done in this section. We shall see a related, but different, intermediate representation in Section 5.7.

This application also illustrates that ADTs do have their limitations. For example, the conventions that we have discussed do not provide an easy way to combine Programs 4.2 and 4.3 into a single

Program 4.3 Infix-to-postfix conversion

This program is another example of a pushdown-stack client. In this case, the stack contains characters—we assume that `Item` is defined to be `char` (that is, we use a different `Item.h` file than for Program 4.2). To convert `(A+B)` to the postfix form `AB+`, we ignore the left parenthesis, convert `A` to postfix, save the `+` on the stack, convert `B` to postfix, then, on encountering the right parenthesis, pop the stack and output the `+`.

```
#include <stdio.h>
#include <string.h>
#include "Item.h"
#include "STACK.h"
main(int argc, char *argv[])
  { char *a = argv[1]; int i, N = strlen(a);
    STACKinit(N);
    for (i = 0; i < N; i++)
      {
        if (a[i] == ')')
          printf("%c ", STACKpop());
        if ((a[i] == '+') || (a[i] == '*'))
          STACKpush(a[i]);
        if ((a[i] >= '0') && (a[i] <= '9'))
          printf("%c ", a[i]);
      }
    printf("\n");
  }
```

program, using the same pushdown-stack ADT for both. Not only do we need two different stacks, but also one of the stacks holds single characters (operators), whereas the other holds numbers. To better appreciate the problem, suppose that the numbers are, say, floating-point numbers, rather than integers. Using a general mechanism to allow sharing the same implementation between both stacks (an extension of the approach that we consider in Section 4.8) is likely to be more trouble than simply using two different stacks (see Exercise 4.16). In fact, as we shall see, this solution might be the approach of choice, because different implementations may have different performance characteristics, so we might not wish to decide a priori that one ADT will serve

both purposes. Indeed, our main focus is on the implementations and their performance, and we turn now to those topics for pushdown stacks.

Exercises

▷4.9 Convert to postfix the expression

```
( 5 * ( ( 9 * 8 ) + ( 7 * ( 4 + 6 ) ) ) ) .
```

▷4.10 Give, in the same manner as Figure 4.2, the contents of the stack as the following expression is evaluated by Program 4.2

```
5 9 * 8 7 4 6 + * 2 1 3 * + * + * .
```

▷4.11 Extend Programs 4.2 and 4.3 to include the - (subtract) and / (divide) operations.

4.12 Extend your solution to Exercise 4.11 to include the unary operators - (negation) and $ (square root). Also, modify the abstract stack machine in Program 4.2 to use floating point. For example, given the expression

```
(-(-1) + $((-1) * (-1)-(4 * (-1))))/2
```

your program should print the value 1.618034.

4.13 Write a PostScript program that draws this figure:

•4.14 Prove by induction that Program 4.2 correctly evaluates any postfix expression.

○4.15 Write a program that converts a postfix expression to infix, using a pushdown stack.

•4.16 Combine Program 4.2 and Program 4.3 into a single module that uses two different stack ADTs: a stack of integers and a stack of operators.

••4.17 Implement a compiler and interpreter for a programming language where each program consists of a single arithmetic expression preceded by a sequence of assignment statements with arithmetic expressions involving integers and variables named with single lower-case characters. For example, given the input

```
(x = 1)
(y = (x + 1))
(((x + y) * 3) + (4 * x))
```

your program should print the value 13.

4.4 Stack ADT Implementations

In this section, we consider two implementations of the stack ADT: one using arrays and one using linked lists. The implementations are

both straightforward applications of the basic tools that we covered in Chapter 3. They differ only, we expect, in their performance characteristics.

If we use an array to represent the stack, each of the functions declared in Program 4.1 is trivial to implement, as shown in Program 4.4. We put the items in the array precisely as diagrammed in Figure 4.1, keeping track of the index of the top of the stack. Doing the *push* operation amounts to storing the item in the array position indicated by the top-of-stack index, then incrementing the index; doing the *pop* operation amounts to decrementing the index, then returning the item that it designates. The *initialize* operation involves allocating an array of the indicated size, and the *test if empty* operation involves checking whether the index is 0. Compiled together with a client program such as Program 4.2 or Program 4.3, this implementation provides an efficient and effective pushdown stack.

We know one potential drawback to using an array representation: As is usual with data structures based on arrays, we need to know the maximum size of the array before using it, so that we can allocate memory for it. In this implementation, we make that information an argument to the function that implements *initialize*. This constraint is an artifact of our choice to use an array implementation; it is not an essential part of the stack ADT. We may have no easy way to estimate the maximum number of elements that our program will be putting on the stack: If we choose an arbitrarily high value, this implementation will make inefficient use of space, and that may be undesirable in an application where space is a precious resource. If we choose too small a value, our program might not work at all. By using an ADT, we make it possible to consider other alternatives, in other implementations, without changing any client program.

For example, to allow the stack to grow and shrink gracefully, we may wish to consider using a linked list, as in the implementation in Program 4.5. In this program, we keep the stack in reverse order from the array implementation, from most recently inserted element to least recently inserted element, to make the basic stack operations easier to implement, as illustrated in Figure 4.5. To *pop*, we remove the node from the front of the list and return its item; to *push*, we create a new node and add it to the front of the list. Because all linked-list operations are at the beginning of the list, we do not need to use a

Program 4.4 Array implementation of a pushdown stack

When there are `N` items in the stack, this implementation keeps them in `s[0], ..., s[N-1]`; in order from least recently inserted to most recently inserted. The top of the stack (the position where the next item to be pushed will go) is `s[N]`. The client program passes the maximum number of items expected on the stack as the argument to `STACKinit`, which allocates an array of that size, but this code does not check for errors such as pushing onto a full stack (or popping an empty one).

```
#include <stdlib.h>
#include "Item.h"
#include "STACK.h"
static Item *s;
static int N;
void STACKinit(int maxN)
  { s = malloc(maxN*sizeof(Item)); N = 0; }
int STACKempty()
  { return N == 0; }
void STACKpush(Item item)
  { s[N++] = item; }
Item STACKpop()
  { return s[--N]; }
```

head node. This implementation does not need to use the argument to `STACKinit`.

Programs 4.4 and 4.5 are two different implementations for the same ADT. We can substitute one for the other without making *any* changes in client programs such as the ones that we examined in Section 4.3. They differ in only their performance characteristics—the time and space that they use. For example, the list implementation uses more time for push and pop operations, to allocate memory for each *push* and deallocate memory for each *pop*. If we have an application where we perform these operations a huge number of times, we might prefer the array implementation. On the other hand, the array implementation uses the amount of space necessary to hold the maximum number of items expected throughout the computation, while the list implementation uses space proportional to the number of items,

Program 4.5 Linked-list implementation of a pushdown stack

This code implements the stack ADT as illustrated in Figure 4.5. It uses an auxiliary function NEW to allocate the memory for a node, set its fields from the function arguments, and return a link to the node.

```
#include <stdlib.h>
#include "Item.h"
typedef struct STACKnode* link;
struct STACKnode { Item item; link next; };
static link head;
link NEW(Item item, link next)
  { link x = malloc(sizeof *x);
    x->item = item; x->next = next;
    return x;
  }
void STACKinit(int maxN)
  { head = NULL; }
int STACKempty()
  { return head == NULL; }
STACKpush(Item item)
  { head = NEW(item, head); }
Item STACKpop()
  { Item item = head->item;
    link t = head->next;
    free(head); head = t;
    return item;
  }
```

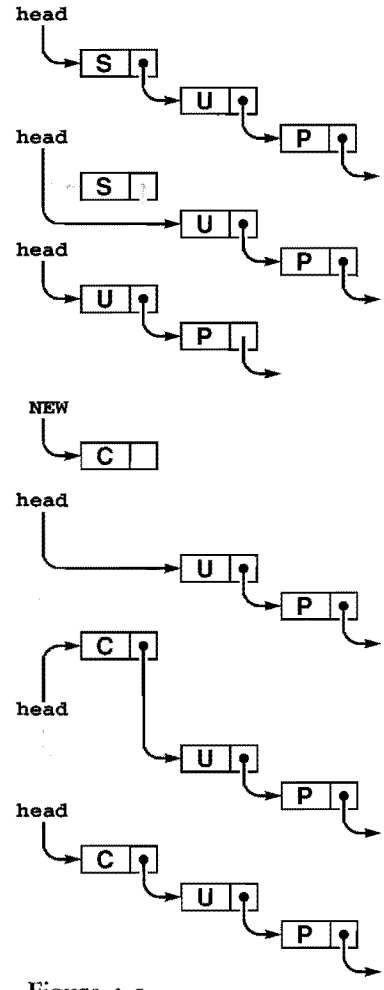


Figure 4.5 Linked-list pushdown stack

The stack is represented by a pointer head, *which points to the first (most recently inserted) item. To* pop *the stack* (top), *we remove the item at the front of the list, by setting* head *from its link. To* push *a new item onto the stack* (bottom), *we link it in at the beginning by setting its link field to* head, *then setting* head *to point to it.*

but always uses extra space for on link per item. If we need a huge stack that is usually nearly full, we might prefer the array implementation; if we have a stack whose size varies dramatically and other data structures that could make use of the space not being used when the stack has only a few items in it, we might prefer the list implementation.

These same considerations about space usage hold for many ADT implementations, as we shall see throughout the book. We often are in the position of choosing between the ability to access any item quickly but having to predict the maximum number of items needed ahead of time (in an array implementation) and the flexibility of always using

space proportional to the number of items in use while giving up the ability to access every item quickly (in a linked-list implementation).

Beyond basic space-usage considerations, we normally are most interested in performance differences among ADT implementations that relate to running time. In this case, there is little difference between the two implementations that we have considered.

Property 4.1 *We can implement the* **push** *and* **pop** *operations for the pushdown stack ADT in constant time, using either arrays or linked lists.*

This fact follows immediately from inspection of Programs 4.4 and 4.5.

■

That the stack items are kept in different orders in the array and the linked-list implementations is of no concern to the client program. The implementations are free to use any data structure whatever, as long as they maintain the illusion of an abstract pushdown stack. In both cases, the implementations are able to create the illusion of an efficient abstract entity that can perform the requisite operations with just a few machine instructions. Throughout this book, our goal is to find data structures and efficient implementations for other important ADTs.

The linked-list implementation supports the illusion of a stack that can grow without bound. Such a stack is impossible in practical terms: at some point, `malloc` will return `NULL` when the request for more memory cannot be satisfied. It is also possible to arrange for an array-based stack to grow dynamically, by doubling the size of the array when the stack becomes half full, and halving the size of the array when the stack becomes half empty. We leave the details of this implementation as an exercise in Chapter 14, where we consider the process in detail for a more advanced application.

Exercises

▷ **4.18** Give the contents of `s[0]`, ... , `s[4]` after the execution of the operations illustrated in Figure 4.1, using Program 4.4.

○ **4.19** Suppose that you change the pushdown-stack interface to replace *test if empty* by *count*, which should return the number of items currently in the data structure. Provide implementations for *count* for the array representation (Program 4.4) and the linked-list representation (Program 4.5).

4.20 Modify the array-based pushdown-stack implementation in the text (Program 4.4) to call a function `STACKerror` if the client attempts to *pop* when the stack is empty or to *push* when the stack is full.

4.21 Modify the linked-list–based pushdown-stack implementation in the text (Program 4.5) to call a function `STACKerror` if the client attempts to *pop* when the stack is empty or if there is no memory available from `malloc` for a *push*.

4.22 Modify the linked-list–based pushdown-stack implementation in the text (Program 4.5) to use an array of indices to implement the list (see Figure 3.4).

4.23 Write a linked-list–based pushdown-stack implementation that keeps items on the list in order from least recently inserted to most recently inserted. You will need to use a doubly linked list.

• 4.24 Develop an ADT that provides clients with two different pushdown stacks. Use an array implementation. Keep one stack at the beginning of the array and the other at the end. (If the client program is such that one stack grows while the other one shrinks, this implementation uses less space than other alternatives.)

• 4.25 Implement an infix-expression–evaluation function for integers that includes Programs 4.2 and 4.3, using your ADT from Exercise 4.24.

4.5 Creation of a New ADT

Sections 4.2 through 4.4 present a complete example of C code that captures one of our most important abstractions: the pushdown stack. The *interface* in Section 4.2 defines the basic operations; *client programs* such as those in Section 4.3 can use those operations without dependence on how the operations are implemented; and *implementations* such as those in Section 4.4 provide the necessary concrete representation and program code to realize the abstraction.

To design a new ADT, we often enter into the following process. Starting with the task of developing a client program to solve an applications problem, we identify operations that seem crucial: What would we *like* to be able to do with our data? Then, we define an interface and write client code to test the hypothesis that the existence of the ADT would make it easier for us to implement the client program. Next, we consider the idea of whether or not we *can* implement the operations in the ADT with reasonable efficiency. If we cannot, we perhaps can seek to understand the source of the inefficiency and to modify the interface to include operations that are better suited to efficient implementation. These modifications affect the client program, and we modify it accordingly. After a few iterations, we have a

Program 4.6 Equivalence-relations ADT interface

The ADT interface mechanism makes it convenient for us to encode precisely our decision to consider the connectivity algorithm in terms of three abstract operations: *initialize*, *find* whether two nodes are connected, and perform a *union* operation to consider them connected henceforth.

```
void UFinit(int);
 int UFfind(int, int);
void UFunion(int, int);
```

working client program and a working implementation, so we freeze the interface: We adopt a policy of not changing it. At this moment, the development of client programs and the development of implementations are separable: We can write other client programs that use the same ADT (perhaps we write some driver programs that allow us to test the ADT), we can write other implementations, and we can compare the performance of multiple implementations.

In other situations, we might define the ADT first. This approach might involve asking questions such as these: What basic operations would client programs want to perform on the data at hand? Which operations do we know how to implement efficiently? After we develop an implementation, we might test its efficacy on client programs. We might modify the interface and do more tests, before eventually freezing the interface.

In Chapter 1, we considered a detailed example where thinking on an abstract level helped us to find an efficient algorithm for solving a complex problem. We consider next the use of the general approach that we are discussing in this chapter to encapsulate the specific abstract operations that we exploited in Chapter 1.

Program 4.6 defines the interface, in terms of two operations (in addition to *initialize*) that seem to characterize the algorithms that we considered in Chapter 1 for connectivity, at a high abstract level. Whatever the underlying algorithms and data structures, we want to be able to check whether or not two nodes are known to be connected, and to declare that two nodes are connected.

Program 4.7 is a client program that uses the ADT defined in the interface of Program 4.6 to solve the connectivity problem. One benefit

Program 4.7 Equivalence-relations ADT client

The ADT of Program 4.6 separates the connectivity algorithm from the union–find implementation, making that algorithm more accessible.

```
#include <stdio.h>
#include "UF.h"
main(int argc, char *argv[])
  { int p, q, N = atoi(argv[1]);
    UFinit(N);
    while (scanf("%d %d", &p, &q) == 2)
      if (!UFfind(p, q))
        { UFunion(p, q); printf(" %d %d\n", p, q); }
  }
```

of using the ADT is that this program is easy to understand, because it is written in terms of abstractions that allow the computation to be expressed in a natural way.

Program 4.8 is an implementation of the union–find interface defined in Program 4.6 that uses a forest of trees represented by two arrays as the underlying representation of the known connectivity information, as described in Section 1.3. The different algorithms that we considered in Chapter 1 represent different implementations of this ADT, and we can test them as such without changing the client program at all.

This ADT leads to programs that are slightly less efficient than those in Chapter 1 for the connectivity application, because it does not take advantage of the property of that client that every *union* operation is immediately preceded by a *find* operation. We sometimes incur extra costs of this kind as the price of moving to a more abstract representation. In this case, there are numerous ways to remove the inefficiency, perhaps at the cost of making the interface or the implementation more complicated (see Exercise 4.27). In practice, the paths are extremely short (particularly if we use path compression), so the extra cost is likely to be negligible in this case.

The combination of Programs 4.6 through 4.8 is operationally equivalent to Program 1.3, but splitting the program into three parts is a more effective approach because it

Program 4.8 Equivalence-relations ADT implementation

This implementation of the weighted-quick-union code from Chapter 1, together with the interface of Program 4.6, packages the code in a form that makes it convenient for use in other applications. The implementation uses a local function `find`.

```
#include <stdlib.h>
#include "UF.h"
static int *id, *sz;
void UFinit(int N)
  { int i;
    id = malloc(N*sizeof(int));
    sz = malloc(N*sizeof(int));
    for (i = 0; i < N; i++)
      { id[i] = i; sz[i] = 1; }
  }
static int find(int x)
  { int i = x;
    while (i != id[i]) i = id[i]; return i; }
int UFfind(int p, int q)
  { return (find(p) == find(q)); }
void UFunion(int p, int q)
  { int i = find(p), j = find(q);
    if (i == j) return;
    if (sz[i] < sz[j])
         { id[i] = j; sz[j] += sz[i]; }
    else { id[j] = i; sz[i] += sz[j]; }
  }
```

- Separates the task of solving the high-level (connectivity) problem from the task of solving the low-level (union–find) problem, allowing us to work on the two problems independently
- Gives us a natural way to compare different algorithms and data structures for solving the problem
- Gives us an abstraction that we can use to build other algorithms
- Defines, through the interface, a way to check that the software is operating as expected

- Provides a mechanism that allows us to upgrade to new representations (new data structures or new algorithms) without changing the client program at all

These benefits are widely applicable to many tasks that we face when developing computer programs, so the basic tenets underlying ADTs are widely used.

Exercises

4.26 Modify Program 4.8 to use path compression by halving.

4.27 Remove the inefficiency mentioned in the text by adding an operation to Program 4.6 that combines *union* and *find*, providing an implementation in Program 4.8, and modifying Program 4.7 accordingly.

○ **4.28** Modify the interface (Program 4.6) and implementation (Program 4.8) to provide a function that will return the number of nodes known to be connected to a given node.

4.29 Modify Program 4.8 to use an array of structures instead of parallel arrays for the underlying data structure.

4.6 FIFO Queues and Generalized Queues

The *first-in, first-out (FIFO) queue* is another fundamental ADT that is similar to the pushdown stack, but that uses the opposite rule to decide which element to remove for *delete*. Rather than removing the most recently inserted element, we remove the element that has been in the queue the longest.

Perhaps our busy professor's "in" box *should* operate like a FIFO queue, since the first-in, first-out order seems to be an intuitively fair way to decide what to do next. However, that professor might not ever answer the phone or get to class on time! In a stack, a memorandum can get buried at the bottom, but emergencies are handled when they arise; in a FIFO queue, we work methodically through the tasks, but each has to wait its turn.

FIFO queues are abundant in everyday life. When we wait in line to see a movie or to buy groceries, we are being processed according to a FIFO discipline. Similarly, FIFO queues are frequently used within computer systems to hold tasks that are yet to be accomplished when we want to provide services on a first-come, first-served basis. Another example, which illustrates the distinction between stacks and FIFO

Program 4.9 FIFO queue ADT interface

This interface is identical to the pushdown stack interface of Program 4.1, except for the names of the structure. The two ADTs differ only in the specification, which is not reflected in the code.

```
void QUEUEinit(int);
 int QUEUEempty();
void QUEUEput(Item);
Item QUEUEget();
```

F		F						
I		F	I					
R		F	I	R				
S		F	I	R	S			
*	F	I	R	S				
T		I	R	S	T			
*	I	R	S	T				
I		R	S	T	I			
N		R	S	T	I	N		
*	R	S	T	I	N			
*	S	T	I	N				
*	T	I	N					
F		I	N	F				
I		I	N	F	I			
*	I	N	F	I				
R		N	F	I	R			
S		N	F	I	R	S		
*	N	F	I	R	S			
*	F	I	R	S				
*	I	R	S					
T		R	S	T				
*	R	S	T					
O		S	T	O				
U		S	T	O	U			
T		S	T	O	U	T		
*	S	T	O	U	T			
*	T	O	U	T				
*	O	U	T					
*	U	T						
*	T							

Figure 4.6
FIFO queue example

This list shows the result of the sequence of operations in the left column (top to bottom), where a letter denotes put *and an asterisk denotes* get. *Each line displays the operation, the letter returned for* get *operations, and the contents of the queue in order from least recently inserted to most recently inserted, left to right.*

queues, is a grocery store's inventory of a perishable product. If the grocer puts new items on the front of the shelf and customers take items from the front, then we have a stack discipline, which is a problem for the grocer because items at the back of the shelf may stay there for a very long time and therefore spoil. By putting new items at the back of the shelf, the grocer ensures that the length of time any item has to stay on the shelf is limited by the length of time it takes customers to purchase the maximum number of items that fit on the shelf. This same basic principle applies to numerous similar situations.

Definition 4.3 *A* **FIFO queue** *is an ADT that comprises two basic operations: insert (***put***) a new item, and delete (***get***) the item that was least recently inserted.*

Program 4.9 is the interface for a FIFO queue ADT. This interface differs from the stack interface that we considered in Section 4.2 only in the nomenclature: to a compiler, say, the two interfaces are identical! This observation underscores the fact that the abstraction itself, which programmers normally do not define formally, is the essential component of an ADT. For large applications, which may involve scores of ADTs, the problem of defining them precisely is critical. In this book, we work with ADTs that capture essential concepts that we define in the text, but not in any formal language, other than via specific implementations. To discern the nature of ADTs, we need to consider examples of their use and to examine specific implementations.

Figure 4.6 shows how a sample FIFO queue evolves through a series of *get* and *put* operations. Each *get* decreases the size of the queue by 1 and each *put* increases the size of the queue by 1. In the figure, the items in the queue are listed in the order that they are put on

the queue, so that it is clear that the first item in the list is the one that is to be returned by the *get* operation. Again, in an implementation, we are free to organize the items any way that we want, as long as we maintain the illusion that the items are organized in this way.

To implement the FIFO queue ADT using a linked list, we keep the items in the list in order from least recently inserted to most recently inserted, as diagrammed in Figure 4.6. This order is the reverse of the order that we used for the stack implementation, but allows us to develop efficient implementations of the queue operations. We maintain two pointers into the list: one to the beginning (so that we can *get* the first element), and one to the end (so that we can *put* a new element onto the queue), as shown in Figure 4.7 and in the implementation in Program 4.10.

We can also use an array to implement a FIFO queue, although we have to exercise care to keep the running time constant for both the *put* and *get* operations. That performance goal dictates that we can not move the elements of the queue within the array, unlike what might be suggested by a literal interpretation of Figure 4.6. Accordingly, as we did with the linked-list implementation, we maintain two indices into the array: one to the beginning of the queue and one to the end of the queue. We consider the contents of the queue to be the elements between the indices. To *get* an element, we remove it from the beginning (head) of the queue and increment the head index; to *put* an element, we add it to the end (tail) of the queue and increment the tail index. A sequence of *put* and *get* operations causes the queue to appear to move through the array, as illustrated in Figure 4.8. When it hits the end of the array, we arrange for it to wrap around to the beginning. The details of this computation are in the code in Program 4.11.

Property 4.2 *We can implement the* **get** *and* **put** *operations for the FIFO queue ADT in constant time, using either arrays or linked lists.*

This fact is immediately clear when we inspect the code in Programs 4.10 and 4.11. ▪

The same considerations that we discussed in Section 4.4 apply to space resources used by FIFO queues. The array representation requires that we reserve enough space for the maximum number of items expected throughout the computation, whereas the linked-list

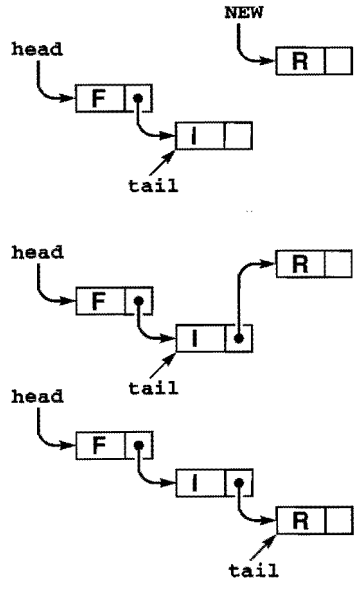


Figure 4.7
Linked-list queue

In this linked-list representation of a queue, we insert new items at the end, so the items in the linked list are in order from least recently inserted to most recently inserted, from beginning to end. The queue is represented by two pointers `head` *and* `tail` *which point to the first and final item, respectively. To* get *an item from the queue, we remove the item at the front of the list, in the same way as we did for stacks (see Figure 4.5). To* put *a new item onto the queue, we set the link field of the node referenced by* `tail` *to point to it* (center), *then update* `tail` (bottom).

Program 4.10 FIFO queue linked-list implementation

The difference between a FIFO queue and a pushdown stack (Program 4.5) is that new items are inserted at the end, rather than the beginning.

Accordingly, this program keeps a pointer `tail` to the last node of the list, so that the function `QUEUEput` can add a new node by linking that node to the node referenced by `tail` and then updating `tail` to point to the new node. The functions `QUEUEget`, `QUEUEinit`, and `QUEUEempty` are all identical to their counterparts for the linked-list pushdown-stack implementation of Program 4.5.

```
#include <stdlib.h>
#include "Item.h"
#include "QUEUE.h"
typedef struct QUEUEnode* link;
struct QUEUEnode { Item item; link next; };
static link head, tail;
link NEW(Item item, link next)
  { link x = malloc(sizeof *x);
    x->item = item; x->next = next;
    return x;
  }
void QUEUEinit(int maxN)
  { head = NULL; }
int QUEUEempty()
  { return head == NULL; }
QUEUEput(Item item)
  {
    if (head == NULL)
      { head = (tail = NEW(item, head)); return; }
    tail->next = NEW(item, tail->next);
    tail = tail->next;
  }
Item QUEUEget()
  { Item item = head->item;
    link t = head->next;
    free(head); head = t;
    return item;
  }
```

Program 4.11 FIFO queue array implementation

The contents of the queue are all the elements in the array between `head` and `tail`, taking into account the wraparound back to 0 when the end of the array is encountered. If `head` and `tail` are equal, then we consider the queue to be empty; but if `put` would make them equal, then we consider it to be full. As usual, we do not check such error conditions, but we make the size of the array 1 greater than the maximum number of elements that the client expects to see in the queue, so that we could augment this program to make such checks.

```
#include <stdlib.h>
#include "Item.h"
static Item *q;
static int N, head, tail;
void QUEUEinit(int maxN)
  { q = malloc((maxN+1)*sizeof(Item));
    N = maxN+1; head = N; tail = 0; }
int QUEUEempty()
  { return head % N == tail; }
void QUEUEput(Item item)
  { q[tail++] = item; tail = tail % N; }
Item QUEUEget()
  { head = head % N; return q[head++]; }
```

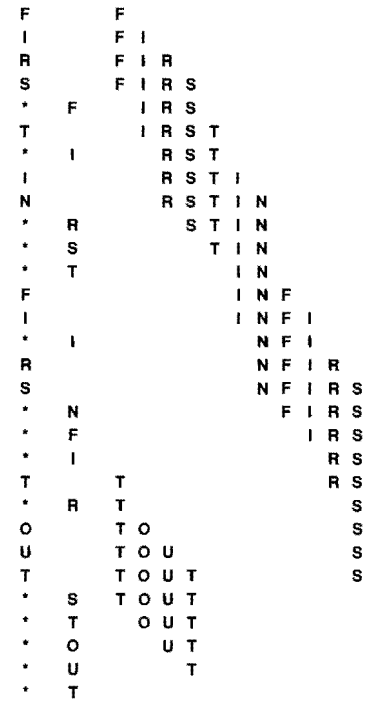


Figure 4.8
FIFO queue example, array implementation

This sequence shows the data manipulation underlying the abstract representation in Figure 4.6 when we implement the queue by storing the items in an array, keeping indices to the beginning and end of the queue, and wrapping the indices back to the beginning of the array when they reach the end of the array. In this example, the tail index wraps back to the beginning when the second T *is inserted, and the head index wraps when the second* S *is removed.*

representation uses space proportional to the number of elements in the data structure, at the cost of extra space for the links and extra time to allocate and deallocate memory for each operation.

Although we encounter stacks more often than we encounter FIFO queues, because of the fundamental relationship between stacks and recursive programs (see Chapter 5), we shall also encounter algorithms for which the queue is the natural underlying data structure. As we have already noted, one of the most frequent uses of queues and stacks in computational applications is to postpone computation. Although many applications that involve a queue of pending work operate correctly no matter what rule is used for *delete*, the overall running time or other resource usage may be dependent on the rule. When such applications involve a large number of *insert* and *delete* operations on data structures with a large number of items on them,

performance differences are paramount. Accordingly, we devote a great deal of attention in this book to such ADTs. If we ignored performance, we could formulate a single ADT that encompassed *insert* and *delete*; since we do not ignore performance, each rule, in essence, constitutes a different ADT. To evaluate the effectiveness of a particular ADT, we need to consider two costs: the implementation cost, which depends on our choice of algorithm and data structure for the implementation; and the cost of the particular decision-making rule in terms of effect on the performance of the client. To conclude this section, we will describe a number of such ADTs, which we will be considering in detail throughout the book.

Specifically, pushdown stacks and FIFO queues are special instances of a more general ADT: the *generalized queue*. Instances of generalized queues differ in only the rule used when items are removed. For stacks, the rule is "remove the item that was most recently inserted"; for FIFO queues, the rule is "remove the item that was least recently inserted"; and there are many other possibilities, a few of which we now consider.

A simple but powerful alternative is the *random queue*, where the rule is to "remove a random item," and the client can expect to get any of the items on the queue with equal probability. We can implement the operations of a random queue in constant time using an array representation (see Exercise 4.42). As do stacks and FIFO queues, the array representation requires that we reserve space ahead of time. The linked-list alternative is less attractive than it was for stacks and FIFO queues, however, because implementing both insertion and deletion efficiently is a challenging task (see Exercise 4.43). We can use random queues as the basis for randomized algorithms, to avoid, with high probability, worst-case performance scenarios (see Section 2.7).

We have described stacks and FIFO queues by identifying items according to the time that they were inserted into the queue. Alternatively, we can describe these abstract concepts in terms of a sequential listing of the items in order, and refer to the basic operations of inserting and deleting items from the beginning and the end of the list. If we insert at the end and delete at the end, we get a stack (precisely as in our array implementation); if we insert at the beginning and delete at the beginning, we also get a stack (precisely as in our linked-list implementation); if we insert at the end and delete at the beginning, we get a

FIFO queue (precisely as in our linked-list implementation); and if we insert at the beginning and delete at the end, we also get a FIFO queue (this option does not correspond to any of our implementations—we could switch our array implementation to implement it precisely, but the linked-list implementation is not suitable because of the need to back up the pointer to the end when we remove the item at the end of the list). Building on this point of view, we are led to the *deque* ADT, where we allow either insertion or deletion at either end. We leave the implementations for exercises (see Exercises 4.37 through 4.41), noting that the array-based implementation is a straightforward extension of Program 4.11, and that the linked-list implementation requires a doubly linked list, unless we restrict the deque to allow deletion at only one end.

In Chapter 9, we consider *priority queues*, where the items have keys and the rule for deletion is "remove the item with the smallest key." The priority-queue ADT is useful in a variety of applications, and the problem of finding efficient implementations for this ADT has been a research goal in computer science for many years. Identifying and using the ADT in applications has been an important factor in this research: we can get an immediate indication whether or not a new algorithm is correct by substituting its implementation for an old implementation in a huge, complex application and checking that we get the same result. Moreover, we get an immediate indication whether a new algorithm is more efficient than an old one by noting the extent to which substituting the new implementation improves the overall running time. The data structures and algorithms that we consider in Chapter 9 for solving this problem are interesting, ingenious, and effective.

In Chapters 12 through 16, we consider *symbol tables*, which are generalized queues where the items have keys and the rule for deletion is "remove an item whose key is equal to a given key, if there is one." This ADT is perhaps the most important one that we consider, and we shall examine dozens of implementations.

Each of these ADTs also give rise to a number of related, but different, ADTs that suggest themselves as an outgrowth of careful examination of client programs and the performance of implementations. In Sections 4.7 and 4.8, we consider numerous examples of

changes in the specification of generalized queues that lead to yet more different ADTs, which we shall consider later in this book.

Exercises

▷4.30 Give the contents of `q[0]`, ..., `q[4]` after the execution of the operations illustrated in Figure 4.6, using Program 4.11. Assume that `maxN` is 10, as in Figure 4.8.

▷4.31 A letter means *put* and an asterisk means *get* in the sequence

E A S * Y * Q U E * * * S T * * * I O * N * * *.

Give the sequence of values returned by the *get* operations when this sequence of operations is performed on an initially empty FIFO queue.

4.32 Modify the array-based FIFO queue implementation in the text (Program 4.11) to call a function `QUEUEerror` if the client attempts to *get* when the queue is empty or to *put* when the queue is full.

4.33 Modify the linked-list–based FIFO queue implementation in the text (Program 4.10) to call a function `QUEUEerror` if the client attempts to *get* when the queue is empty or if there is no memory available from `malloc` for a *put*.

▷4.34 An uppercase letter means *put* at the beginning, a lowercase letter means *put* at the end, a plus sign means *get* from the beginning, and an asterisk means *get* from the end in the sequence

E A s + Y + Q U E * * + s t + * + I O * n + + *.

Give the sequence of values returned by the *get* operations when this sequence of operations is performed on an initially empty deque.

▷4.35 Using the conventions of Exercise 4.34, give a way to insert plus signs and asterisks in the sequence E a s Y so that the sequence of values returned by the *get* operations is (*i*) E s a Y ; (*ii*) Y a s E ; (*iii*) a Y s E ; (*iv*) a s Y E ; or, in each instance, prove that no such sequence exists.

•4.36 Given two sequences, give an algorithm for determining whether or not it is possible to add plus signs and asterisks to make the first produce the second when interpreted as a sequence of deque operations in the sense of Exercise 4.35.

▷4.37 Write an interface for the deque ADT.

4.38 Provide an implementation for your deque interface (Exercise 4.37) that uses an array for the underlying data structure.

4.39 Provide an implementation for your deque interface (Exercise 4.37) that uses a doubly linked list for the underlying data structure.

4.40 Provide an implementation for the FIFO queue interface in the text (Program 4.9) that uses a circular list for the underlying data structure.

4.41 Write a client that tests your deque ADTs (Exercise 4.37) by reading, as the first argument on the command line, a string of commands like those given in Exercise 4.34 then performing the indicated operations. Add a function `DQdump` to the interface and implementations, and print out the contents of the deque after each operation, in the style of Figure 4.6.

○ **4.42** Build a random-queue ADT by writing an interface and an implementation that uses an array as the underlying data structure. Make sure that each operation takes constant time.

•• **4.43** Build a random-queue ADT by writing an interface and an implementation that uses a linked list as the underlying data structure. Provide implementations for *insert* and *delete* that are as efficient as you can make them, and analyze their worst-case cost.

▷ **4.44** Write a client that picks numbers for a lottery by putting the numbers 1 through 99 on a random queue, then prints the result of removing five of them.

4.45 Write a client that takes an integer N from the first argument on the command line, then prints out N poker hands, by putting N items on a random queue (see Exercise 4.4), then printing out the result of picking five cards at a time from the queue.

• **4.46** Write a program that solves the connectivity problem by inserting all the pairs on a random queue and then taking them from the queue, using the quick-find–weighted algorithm (Program 1.3).

4.7 Duplicate and Index Items

For many applications, the abstract items that we process are *unique*, a quality that leads us to consider modifying our idea of how stacks, FIFO queues, and other generalized ADTs should operate. Specifically, in this section, we consider the effect of changing the specifications of stacks, FIFO queues, and generalized queues to disallow duplicate items in the data structure.

For example, a company that maintains a mailing list of customers might want to try to grow the list by performing *insert* operations from other lists gathered from many sources, but would not want the list to grow for an *insert* operation that refers to a customer already on the list. We shall see that the same principle applies in a variety of applications. For another example, consider the problem of routing a message through a complex communications network. We might try going through several paths simultaneously in the network,

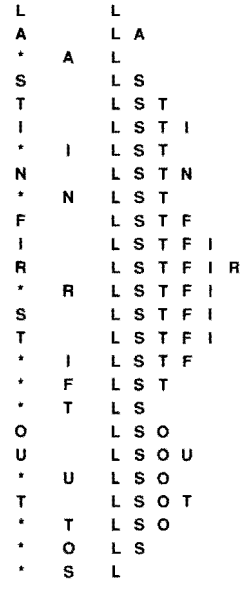


Figure 4.9
Pushdown stack with no duplicates

This sequence shows the result of the same operations as those in Figure 4.1, but for a stack with no duplicate objects allowed. The gray squares mark situations where the stack is left unchanged because the item to be pushed is already on the stack. The number of items on the stack is limited by the number of possible distinct items.

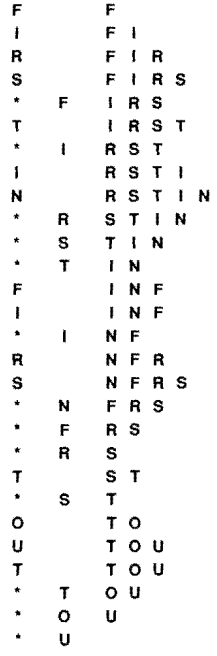


Figure 4.10
FIFO queue with no duplicates, ignore-the-new-item policy

This sequence shows the result of the same operations as those in Figure 4.6, but for a queue with no duplicate objects allowed. The gray squares mark situations where the queue is left unchanged because the item to be put onto the queue is already there.

but there is only one message, so any particular node in the network would want to have only one copy in its internal data structures.

One approach to handling this situation is to leave up to the clients the task of ensuring that duplicate items are not presented to the ADT, a task that clients presumably might carry out using some different ADT. But since the purpose of an ADT is to provide clients with clean solutions to applications problems, we might decide that detecting and resolving duplicates is a part of the problem that the ADT should help to solve.

The policy of disallowing duplicate items is a change in the *abstraction*: the interface, names of the operations, and so forth for such an ADT are the same as those for the corresponding ADT without the policy, but the behavior of the implementation changes in a fundamental way. In general, whenever we modify the specification of an ADT, we get a completely new ADT—one that has completely different properties. This situation also demonstrates the precarious nature of ADT specification: Being sure that clients and implementations adhere to the specifications in an interface is difficult enough, but enforcing a high-level policy such as this one is another matter entirely. Still, we are interested in algorithms that do so because clients can exploit such properties to solve problems in new ways, and implementations can take advantage of such restrictions to provide more efficient solutions.

Figure 4.9 shows how a modified no-duplicates stack ADT would operate for the example corresponding to Figure 4.1; Figure 4.10 shows the effect of the change for FIFO queues.

In general, we have a policy decision to make when a client makes an *insert* request for an item that is already in the data structure. Should we proceed as though the request never happened, or should we proceed as though the client had performed a *delete* followed by an *insert*? This decision affects the order in which items are ultimately processed for ADTs such as stacks and FIFO queues (see Figure 4.11), and the distinction is significant for client programs. For example, the company using such an ADT for a mailing list might prefer to use the new item (perhaps assuming that it has more up-to-date information about the customer), and the switching mechanism using such an ADT might prefer to ignore the new item (perhaps it has already taken steps to send along the message). Furthermore, this policy choice affects the implementations: the forget-the-old-item policy is generally more